

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное
Образовательное учреждение высшего образования
МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
(МОСКОВСКИЙ ПОЛИТЕХ)

ДОПУЩЕН К ЗАЩИТЕ

Заведующий кафедрой

_____ / Пухова Е. А. /

Руководитель образовательной программы

_____ / Даньшина М. В. /

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

по теме:

**РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ ДЛЯ ПЛАНИРОВАНИЯ ЗАДАЧ С
УЧЁТОМ ИХ МЕСТОПОЛОЖЕНИЯ И ВРЕМЕННЫХ ПАРАМЕТРОВ**

по направлению 09.03.01 Информатика и вычислительная техника
Образовательная программа (профиль) «Системная и программная
инженерия»

Студент: _____ / Тютичкин Семен Владимирович, 221–329 /
подпись *ФИО, группа*

Руководитель ВКР: _____ / Васильев Денис Борисович /
подпись *ФИО, уч. звание и степень*

Москва 2026

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное
Образовательное учреждение высшего образования
МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
(МОСКОВСКИЙ ПОЛИТЕХ)

ЗАДАНИЕ НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ
по направлению 09.03.01 Информатика и вычислительная техника
Образовательная программа (профиль) «Системная и программная
инженерия»

Тема ВКР	Разработка веб-приложения для планирования задач с учётом их местоположения и временных параметров
ПРАКТИЧЕСКИЙ РЕЗУЛЬТАТ	
Назначение	Автоматизированное формирование оптимальной последовательности выполнения задач с учетом географического положения, временных окон и длительности выполнения.
Основные функции	1. Управление задачами: создать, изменить, удалить 2. Оптимизация плана выполнения задач 3. Web-интерфейс
Используемые технологии и платформы	Golang, JavaScript, React, Docker, PostgreSQL

ВЫПОЛНЕНИЕ РАБОТЫ	
Решаемые задачи	1. Исследовать проблему оптимизации выполнения задач с гео-параметрами и временными параметрами 2. Проанализировать существующие продукты для оптимизации выполнения задач с гео-параметрами и временными параметрами 3. Разработать свое приложение на основе анализа существующих продуктов
Состав технической документации	1. Список документации
Состав графической части	1. Список графической части

ПЛАН РАБОТЫ НАД ВКР

Этапы	Недели семестра																	
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
Анализ предметной области																		
Выбор технологий для решения проблем предметной области																		
Проектирование системы																		
Разработка веб-приложения																		
Тестирование веб-приложения																		
Конфигурация развертывания системы																		

РУКОВОДИТЕЛЬ ОП:

«___»_____2026, _____ / Даньшина Марина Владимировна /
подпись *ФИО, уч. звание и степень*

РУКОВОДИТЕЛЬ ВКР:

«___»_____2026, _____ / Васильев Денис Борисович /
подпись *ФИО, уч. звание и степень*

СТУДЕНТ:

«___»_____2026, _____ / Тютичкин Семен Владимирович, 221–329/
подпись *ФИО, группа*

АННОТАЦИЯ

Наименование работы: Разработка веб-приложения для автоматизированного планирования маршрутов с учетом временных окон.

Цель работы: разработка веб-приложения для автоматизированного формирования оптимальной последовательности выполнения задач с учетом географического положения, временных окон и длительности выполнения.

Задачи: анализ предметной области и существующих программных решений; исследование алгоритмов маршрутизации с временными окнами; разработка архитектуры программной системы; реализация серверной части и алгоритма оптимизации маршрута; разработка пользовательского интерфейса с визуализацией маршрута, импортом и экспортом задач; интеграция с внешними картографическими сервисами; тестирование и оценка производительности.

Объект исследования — процесс планирования последовательности выполнения задач в городской среде с учетом пространственных и временных параметров. Предмет — алгоритмы маршрутизации и методы оптимизации порядка выполнения задач с временными окнами.

Работа содержит Введение, 3 главы, Заключение и 2 Приложения; включает 10 рисунков, 13 таблиц и библиографический список из 66 источников.

В Первой главе выполнены анализ предметной области, постановка задачи, обоснование выбора технологий и сравнительный анализ аналогов. Во Второй главе представлены модель данных, проект пользовательского интерфейса и техническая реализация, включая алгоритм оптимизации маршрута и руководство по развертыванию. В Третьей главе раскрыты вопросы внедрения и опытной эксплуатации, аспекты отказоустойчивости и производительности. В Заключении приведены выводы и оценка результатов.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	9
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ОБОСНОВАНИЕ ВЫБОРА ТЕХНОЛОГИЙ	11
1.1 Исходное состояние объекта и характеристика предметной области	11
1.2 Постановка проблемы и формирование цели разработки	12
1.3 Обоснование выбора технологий и платформ	13
1.4 Анализ алгоритмических подходов к задаче маршрутизации с временными окнами	16
1.5 Обзор аналогов и сравнительный анализ	19
1.6 Выводы	21
2 РЕАЛИЗАЦИЯ	22
2.1 Модель данных	22
2.2 Архитектура системы	25
2.3 Алгоритм оптимизации маршрута	27
2.4 Тестирование	33
2.5 Интерфейс	37
2.6 Техническая реализация	41
2.7 Руководство пользователя и администратора	46
2.8 Выводы	53
3 ВНЕДРЕНИЕ И ЭКСПЛУАТАЦИЯ	55
3.1 Анализ внедрения продукта	55
3.2 Аспекты отказоустойчивости, надежности и производительности	58
3.3 Внедрение и сопровождение программного продукта	66
3.4 Выводы	68
ЗАКЛЮЧЕНИЕ	70
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	72
ПРИЛОЖЕНИЕ А	80

ПРИЛОЖЕНИЕ Б	87
------------------------	----

ВВЕДЕНИЕ

Рост числа задач, привязанных одновременно к адресу и временному окну, делает ручное планирование маршрута трудоемким и подверженным ошибкам, что формирует потребность в инструментах автоматизированного планирования. Аналогичные тенденции наблюдаются и в коммерческой логистике «последней мили», где увеличение числа адресов доставки и сужение временных окон обслуживания делают задачу построения оптимального маршрута экономически значимой [39]. Применение информационных технологий в управлении временем рассматривается как фактор повышения эффективности планирования личной и профессиональной деятельности [38].

Существующие цифровые решения представлены либо приложениями для управления задачами, ориентированными на учет перечня дел без учета пространственного фактора, либо навигационными сервисами, формирующими маршрут без учета длительности выполнения задач. В результате пользователи вынуждены комбинировать несколько приложений, вручную сопоставляя адреса, интервалы времени и порядок посещения точек. Подобный подход характеризуется высокой трудоемкостью и ростом вероятности ошибок при увеличении количества задач [1].

Объектом исследования является процесс планирования последовательности выполнения задач в городской среде с учетом пространственных и временных параметров.

Предметом исследования являются алгоритмы маршрутизации и методы оптимизации порядка выполнения задач с временными окнами в условиях ограниченных вычислительных ресурсов веб-приложения.

Целью выпускной квалификационной работы является разработка веб-приложения для автоматизированного формирования оптимальной последовательности выполнения задач с учетом географического положения, временных окон и длительности выполнения.

Для достижения поставленной цели необходимо решить следующие задачи:

- проанализировать предметную область и существующие программные решения;
- исследовать алгоритмические подходы к маршрутизации с временными окнами;
- разработать архитектуру программной системы;
- реализовать серверную часть и алгоритм оптимизации маршрута с поддержкой дополнительных ограничений;
- разработать пользовательский интерфейс с визуализацией маршрута на карте, импортом и экспортом задач;
- реализовать интеграцию с внешними картографическими сервисами;
- провести тестирование разработанного решения;
- оценить производительность и масштабируемость системы.

Практическая значимость работы — единый инструмент планирования маршрута с временными окнами, доступный частным пользователям и малому бизнесу без коммерческой подписки.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ОБОСНОВАНИЕ ВЫБОРА ТЕХНОЛОГИЙ

1.1 Исходное состояние объекта и характеристика предметной области

При наличии у задач временных окон (например, «с 9:00 до 12:00») пользователь вынужден вручную сопоставлять время перемещения, длительность выполнения и интервалы доступности каждой точки [13, 50], что многократно повышает трудоемкость планирования с ростом числа задач.

Существующие цифровые инструменты можно разделить на две группы:

- приложения для управления задачами (не учитывают пространственный фактор);
- навигационные сервисы (не работают с временными окнами и списками задач).

Таким образом, пользователь вынужден использовать несколько инструментов одновременно, что увеличивает трудозатраты и повышает вероятность ошибок [1].

Целевой аудиторией системы являются:

- частные пользователи, планирующие личные дела в городе;
- мастера и специалисты на выезде;
- курьеры малого бизнеса;
- индивидуальные предприниматели;
- диспетчеры, формирующие небольшие маршруты вручную.

Особенностью данной предметной области является необходимость одновременного учета:

- географического расположения объектов;
- временных ограничений;

- длительности выполнения задач;
- минимизации времени перемещения и ожидания.

1.2 Постановка проблемы и формирование цели разработки

Анализ исходного состояния показал, что основной проблемой является отсутствие доступного инструмента, объединяющего управление задачами и автоматическую маршрутизацию с учетом временных окон.

Ручной подход:

- требует значительных временных затрат;
- не гарантирует оптимальность маршрута;
- становится неэффективным при увеличении числа точек;
- не обеспечивает формализованного учета временных ограничений.

Целью разработки является создание веб-приложения, обеспечивающего автоматизированное формирование оптимального маршрута выполнения задач с учетом их географического положения, временных окон и длительности выполнения.

На основании цели сформирован следующий перечень функций:

- создание, редактирование, удаление и клонирование задач;
- импорт задач из файлов формата CSV и XLSX (Excel);
- экспорт задач в форматах CSV и XLSX (Excel).
- получение матрицы расстояний через внешние API;
- вычисление оптимальной последовательности выполнения задач;
- поддержка дополнительных ограничений маршрута: фиксация начальной и конечной точки, задание порядка выполнения пар задач;
- выбор способа передвижения (автомобиль, пешком, общественный транспорт);
- расчет времени прибытия, ожидания и завершения для каждой остановки;

- визуальное обнаружение конфликтов временных окон;
- визуализация маршрута на карте;

Для обеспечения соответствия назначению определены следующие нефункциональные требования:

- время ответа сервера — не более 2 секунд при типовой нагрузке 5 запросов в секунду;
- поддержка современных браузеров (Google Chrome версии 148 и выше, Yandex Browser версии 25 и выше);
- масштабируемость до 100 одновременных пользователей;
- устойчивость к повторным запросам внешних API за счет кэширования.

1.3 Обоснование выбора технологий и платформ

1.3.1 Выбор серверной платформы

В качестве языка для написания серверной части был выбран Golang[2].

Выбор языка Golang[2] обусловлен:

- статической типизацией, снижающей риск ошибок;
- встроенной моделью конкурентности на основе горутин [14];
- наличием практического опыта разработки.

1.3.2 Выбор СУБД и работы с геоданными

В качестве СУБД выбрана PostgreSQL[3].

Обоснование выбора:

- наличие геометрических индексов (GiST[4]);
- непрерывное промышленное применение с 1996 года; стабильный релизный цикл и соответствие стандарту SQL:2016 [3];

— наличием практического опыта разработки.

1.3.3 Выбор клиентских технологий

Для реализации пользовательского интерфейса был выбран язык TypeScript[25] с библиотекой React[5].

Причины выбора:

- компонентный подход к разработке;
- развитая экосистема;
- наличие поддержки картографических библиотек.

В проекте применяется TypeScript[25] — статически типизированное надмножество JavaScript[?], что обеспечивает раннее обнаружение ошибок на этапе компиляции. Эмпирическое исследование 604 проектов на GitHub подтверждает, что TypeScript-проекты демонстрируют значительно более высокое качество и понятность кода по сравнению с аналогами на JavaScript [27].

1.3.4 Выбор внешнего картографического сервиса

Для геокодирования адресов и визуализации маршрута выбран сервис «Яндекс JavaScript API и HTTP Геокодер» [22] (далее — Яндекс JS API). В таблице 1 представлено сравнение доступных картографических решений.

Таблица 1 – Сравнение картографических API

Критерий	Яндекс JS API	Google Maps JS API	OpenStreetMap / Leaflet
Геокодер на русском языке	Да, включая топонимику РФ	Да	Качество в РФ ниже коммерческих провайдеров; обновление зависит от сообщества
Бесплатный лимит (геокодер)	1 000 за пр./сутки	200 \$ кредит/мес.	Без ограничений
Работа в России	Без ограничений	Может быть недоступен	Без ограничений

По совокупности критериев таблицы 1 для целевой аудитории (российские пользователи) выбран Яндекс JS API: единственный из рассмотренных вариантов, обеспечивающий поддержку российской топонимики, встроенный мультимаршрутизатор для построения матрицы расстояний и гарантированную доступность в российском сетевом сегменте.

1.3.5 Итоговый технологический стек

В таблице 2 приведен обоснованный итоговый технологический стек.

Таблица 2 – Итоговый технологический стек разрабатываемой системы

Уровень	Технология	Обоснование
Серверная часть	Go 1.23 + Gin [24]	Нативная компиляция, модель конкурентности goroutine
База данных	PostgreSQL 16	Реляционность, геоиндексы GiST, зрелость платформы
Клиентская часть	React + TypeScript [25]	Компонентный подход, ста- тическая типизация, разви- тая экосистема
Сборка клиент- ской части	Vite 6 [26]	Горячая замена модулей, оптимизированная сборка для производственной среды
Картографический API	Яндекс JS API 2.1	Русскоязычный геокодер, встроенный мультимарш- рутизатор
Контейнеризация	Docker Compose [6]	Воспроизводимость среды, автоматическое примене- ние миграций

1.4 Анализ алгоритмических подходов к задаче маршрутизации с временными окнами

Задача, решаемая в данной работе, относится к классу TSPTW, рассмотренному в предыдущем разделе, — обобщению классической задачи коммивояжера (TSP) [18] с дополнительными временными ограничениями.

1.4.1 Формальная постановка задачи

Пусть задан граф $G = (V, E)$, где $V = \{v_0, v_1, \dots, v_n\}$ — множество узлов (задач пользователя), E — множество дуг с матрицей расстояний d_{ij} и матрицей времен перемещения t_{ij} .

Каждый узел v_i ($i \geq 1$) характеризуется:

— временным окном $[a_i, b_i]$ — допустимым интервалом начала обслуживания;

— длительностью обслуживания s_i (минуты).

Если маршрутный агент прибывает в узел v_i в момент $\tau_i < a_i$, он ожидает до момента a_i ; если $\tau_i > b_i - s_i$ — обслуживание начать невозможно (нарушение ограничения).

Цель: найти порядок посещения всех узлов, минимизирующий суммарное время маршрута $T = \sum_i t_{ij} + \sum_i w_i$, где $w_i = \max(0, a_i - \tau_i)$ — время ожидания в узле v_i , при соблюдении условий $\tau_i \leq b_i - s_i$ для всех i .

На рисунке 1 приведен пример графа задачи TSPTW с тремя узлами. Ребра графа подписаны временем перемещения d_{ij} (в минутах), получаемым из матрицы расстояний внешнего API. Алгоритм NNN-TW выбирает оптимальный порядок посещения: $v_0 \rightarrow v_2 \rightarrow v_1$ — посещение ближайшей допустимой точки на каждом шаге.

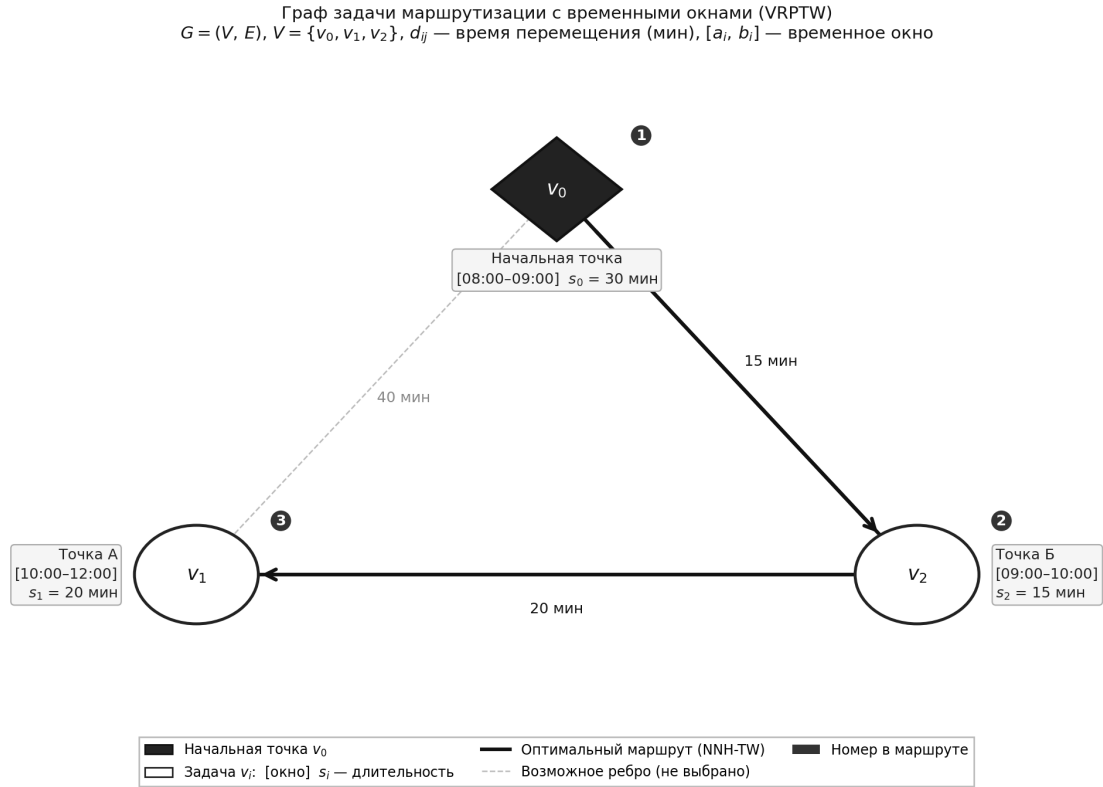


Рисунок 1 – Пример графа задачи коммивояжёра с временными окнами (TSPTW): d_{ij} — время перемещения между узлами (мин), $[a_i, b_i]$ — временное окно, s_i — длительность обслуживания; жирными стрелками выделен оптимальный маршрут $v_0 \rightarrow v_2 \rightarrow v_1$

1.4.2 Сравнение алгоритмических подходов

В таблице 3 приведен сравнительный анализ основных подходов к решению TSPTW применительно к задачам данной работы.

Таблица 3 – Сравнение алгоритмических подходов к задаче TSPTW

Подход	Сложность	Оптимальность маршрута
Полный перебор (brute force) [30]	$O(n!)$	Да
Динамическое программирование (алг. Хелда–Карпа [18])	$O(2^n \cdot n^2)$	Да
Жадная эвристика NNH-TW (ближайший сосед с временными окнами и просмотром вперед)	$O(n^3)$	Нет
Муравьиный алгоритм (ACO) [10, 40]	$O(iter \cdot n^2 \cdot m)$	Нет
Генетические алгоритмы (GA) [19]	$O(gen \cdot pop \cdot n)$	Нет

Здесь n — количество узлов маршрута, m — количество муравьев (агентов), $iter$ — число итераций, gen — число поколений, pop — размер популяции.

1.4.3 Обоснование выбора алгоритма

Для разрабатываемого веб-приложения, ориентированного на частных пользователей и малый бизнес, был выбран алгоритм «ближайшего соседа с временными окнами» (NNH-TW, Nearest Neighbour Heuristic with Time Windows) по следующим причинам:

— **Полиномиальная сложность $O(n^3)$** : на каждом из n шагов основного цикла для каждого из n кандидатов выполняется одношаговый просмотр вперед (look-ahead) по n оставшимся узлам. Экспериментально (см. таблицу 12 в разделе «Производительность алгоритма оптимизации») среднее время рабо-

ты при $n = 30$ составляет несколько микросекунд — на три-четыре порядка меньше задержки получения матрицы расстояний от внешнего API;

— **Предсказуемость**: время работы зависит от числа задач, что гарантирует выполнение нефункционального требования по времени ответа;

— **Качество решений**: жадные конструктивные эвристики обеспечивают приемлемое качество решений при низких вычислительных затратах [48], что достаточно для целей планирования личных задач и курьерских маршрутов малого масштаба;

— **Расширяемость**: реализованный интерфейс `Optimizer` позволяет заменить алгоритм на более мощный (например, АСО или GA) без изменения остального кода системы.

1.5 Обзор аналогов и сравнительный анализ

Существующие программные решения, частично пересекающиеся с задачами разрабатываемой системы, можно разделить на три класса.

Первый класс — приложения для управления задачами (Todoist [42], Microsoft To Do [43]): реализуют полноценное создание, редактирование и удаление задач, поддерживают напоминания и мультиплатформенную синхронизацию, однако полностью лишены функций геокодирования адресов и построения маршрутов.

Второй класс — навигационные сервисы (Google Maps [44], Яндекс карты [?], 2GIS [?]): обеспечивают прокладку маршрутов по нескольким точкам, однако не поддерживают автоматическую оптимизацию порядка посещения и не предоставляют средств управления задачами [45].

Третий класс — специализированные логистические платформы (Routific [46], Relog [7], Яндекс Маршрутизация [8], Spoke [9]): сочетают геокодирование, оптимизацию маршрутов и учет временных окон, однако ориентированы на корпоративное управление автопарком (массовая обработка

заказов, несколько водителей, диспетчеры) и недоступны для индивидуального применения без коммерческой подписки.

В таблице 4 приведен сравнительный анализ представителей всех трех классов.

Таблица 4 – Сравнительный анализ аналогов

Критерий	Todoist	Microsoft To Do	Google Maps	Routific	Relog / Яндекс Маршрутизация	Проектируемое решение
Управление задачами	Да	Да	Нет	Нет	Нет	Да
Геокодирование адресов	Нет	Нет	Да	Да	Да	Да
Автоматическая оптимизация маршрута	Нет	Нет	Нет	Да	Да	Да
Учет временных окон	Нет	Нет	Нет	Да	Нет	Да
Целевой сегмент	Частные пользователи	Частные пользователи	Массовый	Корпоративный	Средний и крупный бизнес	Частный пользователь и малый бизнес
Импорт/экспорт задач (CSV, XLSX)	Нет	Нет	Нет	Да	Да	Да
Модель распространения	Условно-бесплатная	Бесплатный	Бесплатный	Платная с триалом	Платная подписка	Бесплатный

Из таблицы 4 следует, что ни один из рассмотренных аналогов не обеспечивает одновременно управление задачами, автоматическую оптимизацию маршрута с учетом временных окон и доступность для частного пользователя без коммерческой подписки. Менеджеры задач первого класса лишены геолокационных функций; навигационные сервисы второго класса не поддерживают автоматическую оптимизацию порядка посещения и не управляют задачами; логистические платформы третьего класса реализуют требуемый функционал, однако ориентированы на корпоративный автопарк и недоступны для целевой аудитории разрабатываемой системы.

1.6 Выводы

Проведен анализ предметной области, выявлены ограничения существующих инструментов и обоснована актуальность разработки системы планирования маршрутов с учетом временных окон. Сформулирована цель и назначение продукта, определен перечень функций и нефункциональных требований.

Проведен сравнительный анализ алгоритмических подходов к задаче TSPTW: полный перебор, динамическое программирование, жадные эвристики и метаэвристики. Обоснован выбор алгоритма NNN-TW с полиномиальной сложностью $O(n^3)$ как наиболее подходящего для целевого масштаба системы ($n < 50$) и требований по времени ответа.

На основании сравнительного анализа технологий выбран стек разработки, обеспечивающий требуемую производительность и масштабируемость системы. Проведенный анализ аналогов подтвердил целесообразность создания специализированного решения, ориентированного на частных пользователей и малый бизнес.

2 РЕАЛИЗАЦИЯ

2.1 Модель данных

2.1.1 Общая характеристика структуры хранения данных

На рисунке 2 представлена логическая модель данных разрабатываемого веб-приложения.

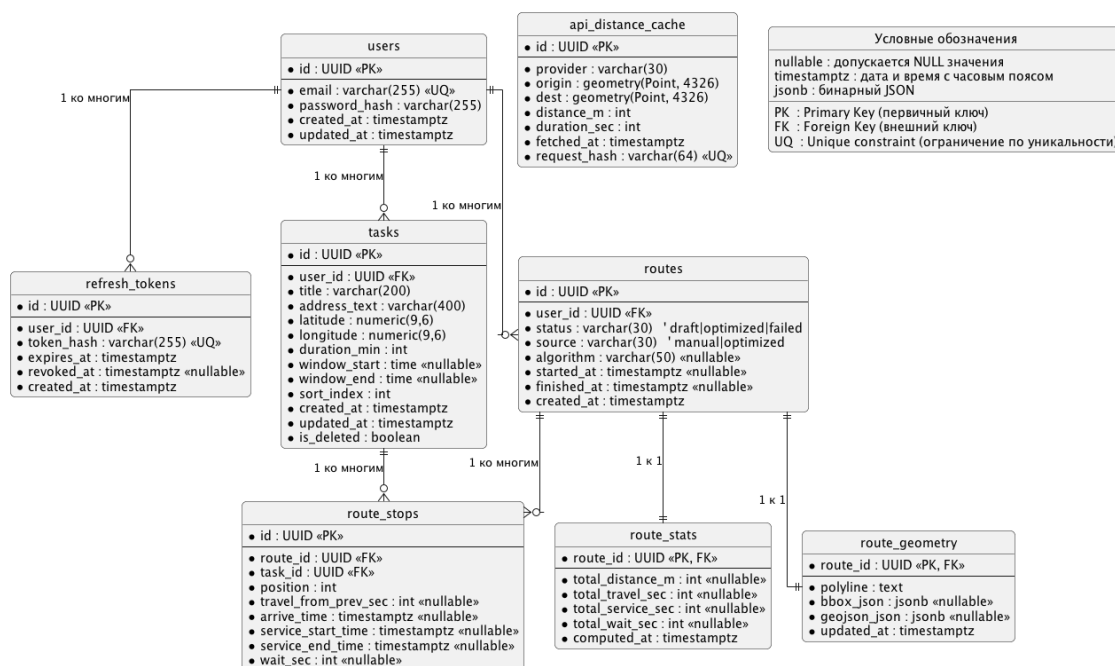


Рисунок 2 – Модель данных

Структура базы данных включает следующие функциональные группы сущностей:

- хранение информации о пользователях;
- хранение задач с географической привязкой;
- хранение маршрутов;
- хранение результатов вычислений.

2.1.2 Сущности, связанные с пользователями

Сущность `users` предназначена для хранения учетных записей пользователей. Первичный ключ `id` используется для установления связей со

всеми зависимыми таблицами. Поле `email` выполняет роль логина и имеет ограничение уникальности, что исключает дублирование учетных записей. Пароль хранится в виде хэшированного значения, что обеспечивает безопасность аутентификации.

Сущность `refresh_tokens` используется для поддержки механизма продления авторизации. Каждый токен связан с конкретным пользователем через внешний ключ `user_id`. Хранение токена в виде хэша предотвращает его компрометацию в случае несанкционированного доступа к базе данных.

2.1.3 Сущность задач

Сущность `tasks` отражает основную предметную сущность системы — задачу, подлежащую выполнению.

Каждая запись содержит:

- связь с владельцем задачи (`user_id`);
- текстовое описание адреса (`address_text`);
- географические координаты (`latitude`, `longitude`), допускающие NULL для задач без адреса;
- длительность выполнения (`duration_min`), допускающую NULL для мгновенных задач;
- отдельные дату и время начала (`window_start_date`, `window_start_time`) и окончания (`window_end_date`, `window_end_time`) временного окна, что позволяет задавать ограничения как по дате, так и по времени;
- индекс сортировки (`sort_index`) для отображения порядка в пользовательском интерфейсе;
- признак выполнения (`is_completed`) для отметки завершённых задач.

2.1.4 Сущности маршрутов

Сущность `routes` описывает факт построения маршрута для конкретного пользователя. Она содержит сведения о статусе вычисления, источнике формирования последовательности (ручной или оптимизированный), пользовательское имя маршрута (`name`) и время создания.

Структура маршрута детализируется в сущности `route_stops`, где каждая запись соответствует отдельной точке маршрута. Поле `position` определяет порядок следования, а временные поля (`arrive_time`, `service_start_time`, `service_end_time`) позволяют зафиксировать результат работы алгоритма с учетом временных окон.

2.1.5 Хранение агрегированных показателей

Агрегированные характеристики маршрута вынесены в отдельную сущность `route_stats`. К таким показателям относятся суммарная длина маршрута, время перемещения, время выполнения задач и суммарное ожидание.

Выделение агрегированных данных в отдельную таблицу позволяет:

- уменьшить нагрузку на основную таблицу маршрутов;
- обновлять статистику независимо от структуры маршрута;
- отделить расчетные показатели от исходных данных.

2.1.6 Геометрия маршрута

Сущность `route_geometry` предназначена для хранения пространственного представления маршрута, используемого при визуализации на карте. Поле `polyline` либо `geojson_json` содержит геометрическое описание линии маршрута, полученное из внешнего API.

Разделение логической структуры маршрута и его геометрического представления позволяет обновлять визуальные данные без изменения структуры остановок.

2.1.7 Кэширование данных внешних сервисов

Результаты обращений к внешним сервисам маршрутизации кэшируются на уровне приложения в оперативной памяти через обёртку `CachedProvider` над интерфейсом `distance.Provider`. В качестве хранилища используется готовая библиотека `hashicorp/golang-lru/v2` [47] (пакет `expirable`), реализующая потокобезопасный LRU-кэш с ограничением размера и временем жизни записи (TTL). Стратегия LRU выбрана как одна из наиболее эффективных и наименее ресурсоемких политик вытеснения для кэшей с ограниченным объемом [36]. Ключом записи служит пара географических координат (округлённых до шестого знака после запятой), что делает кэш инвариантным относительно порядка запросов одних и тех же адресов.

Наличие кэша снижает количество обращений к внешним сервисам, уменьшает задержки при повторной оптимизации и повышает устойчивость системы при ограничениях со стороны API-провайдера.

2.2 Архитектура системы

2.2.1 Общие принципы

Серверная часть приложения построена по принципам чистой архитектуры (Clean Architecture), предложенной Р. Мартином [17]; Ключевой принцип: зависимости направлены исключительно от внешних слоев (HTTP-обработчики, репозитории) к внутренним (доменные сущности и варианты использования), но не наоборот. Это обеспечивает независимость бизнес-

логики от конкретных фреймворков, СУБД и транспортных протоколов. Применимость данного подхода к серверным приложениям на языке Go с реализацией REST API подтверждена прикладной работой [37].

2.2.2 Слои архитектуры

Приложение организовано в четыре слоя, представленных в таблице 5.

Таблица 5 – Слои чистой архитектуры серверной части

Слой	Пакет	Назначение
Доменные сущности	<code>internal/domain/*</code>	Структуры данных (Task, Route, User); без внешних зависимостей
Интерфейсы (порты)	<code>internal/domain/*gate</code>	Интерфейсы репозитория и оптимизатора; определяют контракт без реализации
Варианты использования	<code>internal/domain/*story</code>	Бизнес-логика: создание задач, пакетный импорт (task import), запуск оптимизации, управление сессией
Адаптеры	<code>internal/api/http</code> и <code>internal/platform/postgres</code>	HTTP-обработчики и SQL-репозитории; зависят от внешних библиотек

2.2.3 Паттерн замены алгоритма

Ключевым архитектурным решением является интерфейс `Optimizer`, определенный в пакете `internal/domain/route/gate`:

— метод `Optimize(ctx, graph, startTimeUnix, constraints)` принимает граф задач, время выезда в секундах Unix и структуру дополнительных ограничений (фиксированные начальная/конечная точки, попарные ограничения порядка посещения), возвращает порядок обхода, тайминги каждой остановки и агрегированную статистику;

— конкретная реализация `NearestNeighborTW` регистрируется в модуле сборки зависимостей (`internal/app/app.go`);

— для замены алгоритма (например, на муравьиный или генетический) достаточно зарегистрировать новую реализацию интерфейса, не изменяя HTTP-обработчики, репозитории или доменные сущности.

2.3 Алгоритм оптимизации маршрута

2.3.1 Описание алгоритма

Для вычисления оптимального порядка посещения задач реализован алгоритм «ближайшего соседа с временными окнами» (Nearest Neighbour Heuristic with Time Windows, NNH-TW). Базовый алгоритм ближайшего соседа — одна из наиболее изучаемых конструктивных эвристик для задачи коммивояжера, традиционно используемая в качестве эталона при сравнении более сложных методов [28].

Алгоритм является жадной конструктивной эвристикой с одношаговым просмотром вперед (one-step look-ahead): на каждом шаге из допустимых кандидатов выбирается узел с наименьшим временем завершения обслуживания, при этом предпочтение отдается «безопасным» кандидатам — тем, посещение которых не делает недостижимым ни один другой узел с временным окном. Внутри каждого класса (безопасные / рискованные) приоритет получает кандидат с более ранним дедлайном, что снижает вероятность пропуска срочных задач. Алгоритм также учитывает дополнительные ограничения: фиксацию начальной и конечной точки маршрута, а также попарные ограничения порядка посещения (предшествование).

2.3.2 Псевдокод алгоритма

Листинг 2.1 – Псевдокод алгоритма NNH-TW с просмотром вперед

```

1 Вход: граф  $G(V, E)$ ,  $startTimeUnix$ ,  $constraints\{startIdx,$ 
     $\hookrightarrow endIdx, precedences\}$ 
2 Выход: порядок обхода  $order[]$ , тайминги[], агрегированная
     $\hookrightarrow$  статистика

3 1.  $prereqs \leftarrow buildPrereqs(precedences)$ 
4 2.  $endIdx \leftarrow constraints.endIdx$  (или -1)
5 3. Если  $constraints.startIdx$  задан:
6      $cur \leftarrow constraints.startIdx$ 
7 Иначе:
8      $cur \leftarrow$  узел с наиболее ранним окном среди допустимых
9 4.  $visited[cur] \leftarrow true$ ;  $order \leftarrow [cur]$ 
10 5.  $currentTimeSec \leftarrow startTimeUnix + duration[cur] * 60$ 

11 6. Пока  $|order| < |V|$ :
12     // Если остался только закреплённый конечный узел —
     $\hookrightarrow$  ставим его
13     а. Если  $endIdx \geq 0$  И все прочие посещены:
14         Добавить  $endIdx$  в маршрут; выйти из цикла

15     // Проход 1: допустимый узел с минимальным
     $\hookrightarrow completionTime + look-ahead$ 
16     б.  $bestComp \leftarrow \infty$ ;  $bestSafe \leftarrow false$ ;  $bestDeadline \leftarrow \infty$ ;
     $\hookrightarrow next \leftarrow -1$ 
17     в. Для каждого  $i \notin visited, i \neq endIdx$ :
18         Если не выполнены предшественники  $i$ : пропустить
19          $arrivalSec \leftarrow currentTimeSec + travelSec[cur][i]$ 
20         Если НЕ  $feasible(i, arrivalSec)$ : пропустить
21          $comp \leftarrow completionTime(i, arrivalSec)$ 
22          $safe \leftarrow$  НЕ  $causesWindowMiss(i, comp, G, visited,$ 
     $\hookrightarrow endIdx)$ 
23         Если  $(safe > bestSafe)$  ИЛИ  $(safe = bestSafe$  И  $comp$ 
     $\hookrightarrow < bestComp)$ 

```

```

24         ИЛИ (safe = bestSafe И comp = bestComp И
           ↪ deadline[i] < bestDeadline):
25         next ← i; bestComp ← comp; bestSafe ← safe

26     // Проход 2: резервный – минимальный completionTime без
           ↪ учёта окна
27     d. Если next = -1:
28         Для каждого i ∈ visited, i ≠ endIdx:
29             Если не выполнены предшественники i: пропустить
30             comp ← completionTime(i, arrivalSec)
31             Если comp < bestComp: next ← i

32     e. waitSec ← max(0, windowStart[next] - arrivalSec)
33     f. currentTimeSec ← arrivalSec + waitSec +
           ↪ duration[next] * 60
34     g. visited[next] ← true; order.append(next); cur ←
           ↪ next

35 7. Вернуть order, тайминги и статистику

36 feasible(i, arrivalSec):
37     Если windowEnd[i] < 0: вернуть true
38     Вернуть arrivalSec ≤ windowEnd[i] - duration[i] * 60

39 causesWindowMiss(cand, afterCandSec, G, visited, endIdx):
40     Для каждого j ∈ visited, j ≠ cand, j ≠ endIdx:
41         Если windowEnd[j] < 0: пропустить
42         Если НЕ feasible(j, afterCandSec +
           ↪ travelSec[cand][j]):
43             Вернуть true
44     Вернуть false

```

2.3.3 Блок-схема алгоритма

Блок-схема алгоритма выполнена в соответствии с условными обозначениями ГОСТ 19.701-90 [32] и представлена на рисунке 3.

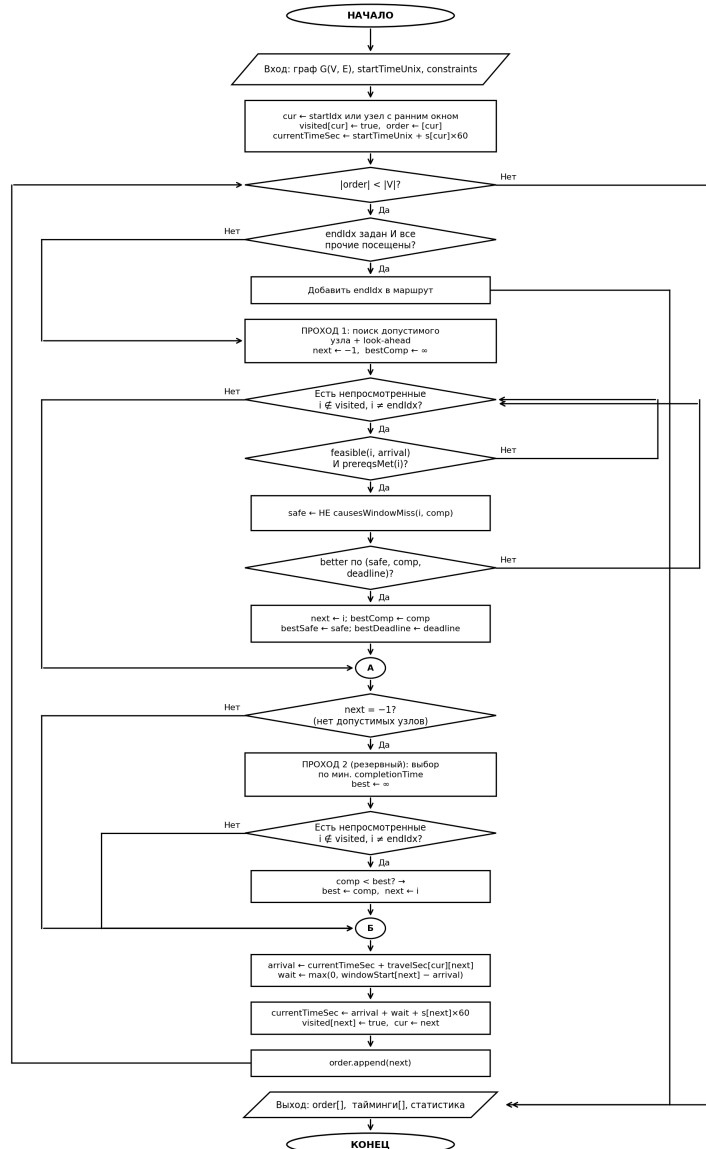


Рисунок 3 – Блок-схема алгоритма ближайшего соседа с временными окнами (NNH-TW)

2.3.4 Трассировка на числовом примере

Рассмотрим пример с тремя задачами для иллюстрации работы алгоритма. Исходные данные приведены в таблице 6, матрица времен перемещения — в таблице 7.

Таблица 6 – Исходные данные задач для трассировки

Узел	Адрес	Временное окно	Длительность
v_0	Офис (начало)	08:00–09:00	30 мин
v_1	Точка А	10:00–12:00	20 мин
v_2	Точка Б	09:00–10:00	15 мин

Таблица 7 – Матрица времен перемещения (минуты)

Из / В	v_0	v_1	v_2
v_0	—	40	15
v_1	40	—	20
v_2	15	20	—

Трассировка алгоритма при начале работы в 08:00 (для наглядности время приведено в минутах от полуночи; в реализации используются секунды Unix):

Выбор стартового узла. Узел v_0 имеет наиболее раннее окно (08:00), поэтому именно он выбирается стартовым: $\text{cur} = 0$, $\text{currentTime} = 480 + 30 = 510$ (08:30 плюс 30 мин выполнения, что соответствует 09:30 или 510 мин от полуночи).

Шаг 1. Проход 1. Для кандидата v_1 прибытие составляет $510 + 40 = 550$ (09:10). Окно v_1 задано как 600–720 (10:00–12:00), условие $550 \leq 720 - 20 = 700$ выполнено, время в пути $t = 40$ мин. Для кандидата v_2 прибытие составляет $510 + 15 = 525$ (08:45), окно 540–600 (09:00–10:00), условие $525 \leq 600 - 15 = 585$ выполнено, $t = 15$ мин. Поскольку v_2 ближе ($15 < 40$), выбирается $\text{next} = 2$. Ожидание открытия окна составляет $\max(0, 540 - 525) = 15$ мин, что даёт $\text{currentTime} = 525 + 15 + 15 = 555$ (09:15).

Шаг 2. Проход 1. Для оставшегося кандидата v_1 прибытие составляет $555 + 20 = 575$ (09:35), условие $575 \leq 700$ выполнено, $\text{next} = 1$. Ожидание равно $\max(0, 600 - 575) = 25$ мин, после чего $\text{currentTime} = 575 + 25 + 20 = 620$ (10:20).

Итоговый порядок: $v_0 \rightarrow v_2 \rightarrow v_1$. Суммарное ожидание: $15 + 25 = 40$ мин.

2.3.5 Оценка вычислительной сложности

Алгоритм выполняет n итераций основного цикла. На каждой итерации для каждого из n кандидатов выполняется одношаговый просмотр вперед (`causesWindowMiss`), просматривающий до n оставшихся узлов. Таким образом, сложность составляет $O(n^3)$.

При $n = 30$ задачах алгоритм выполняет порядка $30^3 = 27\,000$ элементарных операций. Результаты бенчмаркинга, приведенные в таблице 12, показывают, что при $n \leq 30$ время работы алгоритма составляет единицы микросекунд, что на несколько порядков меньше задержки, вносимой получением матрицы расстояний от внешнего API.

2.3.6 Дополнительные ограничения маршрута

Помимо временных окон, реализована поддержка трёх типов дополнительных ограничений, задаваемых пользователем при запросе оптимизации.

Фиксированная начальная точка. Если пользователь указывает идентификатор начальной задачи, алгоритм начинает обход с соответствующего узла вместо автоматически выбранного по признаку наиболее раннего временного окна.

Фиксированная конечная точка. Если задана конечная задача, соответствующий узел исключается из стандартного процесса выбора следующей точки. После того как все остальные узлы посещены, конечный узел добавляется в маршрут последним.

Ограничения порядка. Пользователь вправе указать пары задач вида «А до В»: задача А должна быть выполнена строго раньше задачи В. При выборе следующего узла на каждом шаге алгоритм пропускает кандидатов,

для которых не все предшественники уже посещены. Условие проверяется в обоих проходах (Pass 1 и Pass 2), что гарантирует соблюдение ограничения независимо от наличия допустимых временных окон.

Ограничения передаются в алгоритм через структуру `Constraints` пакета `optimizer`. Преобразование идентификаторов задач в индексы графа выполняется на уровне бизнес-логики (`story.Optimize`) и не затрагивает реализацию алгоритма, что соответствует принципу единственной ответственности.

2.3.7 Вывод

Алгоритм NNH-TW при сложности $O(n^3)$ обеспечивает приемлемое качество решений [48] для целевого масштаба системы ($n < 50$) и расширен поддержкой фиксированных начальной/конечной точек и ограничений порядка выполнения.

2.4 Тестирование

2.4.1 Стратегия тестирования

В проекте реализована многоуровневая стратегия тестирования, охватывающая как серверную, так и клиентскую части приложения. Выбор уровней тестирования соответствует характеристикам качества программного продукта, определенным в [31], а классификация уровней (модульное, интеграционное, компонентное) соответствует общепринятой в [33].

Применяемые виды тестирования:

- **Модульные тесты** — проверка отдельных функций и методов в изоляции от внешних зависимостей;
- **Тесты с заглушками** — тестирование вариантов использования (`story`) через интерфейсы с подмененными реализациями репозиториев;

— **Тесты компонентов** — проверка React-компонентов через `@testing-library/react` с симуляцией пользовательских действий;

— **Тесты API-клиента** — перехват HTTP-запросов через Mock Service Worker (MSW) и проверка логики обновления токена.

Для написания тестов серверной части используется библиотека `github.com/stetchr/testify`: пакет `assert` для нестрогих проверок и пакет `require` для немедленной остановки теста при критическом сбое.

2.4.2 Тестирование алгоритма оптимизации

Наиболее критичным компонентом с точки зрения корректности является алгоритм NNH-TW. Для его проверки разработано 35 тест-кейсов, охватывающих все ветви алгоритма. Основные тест-кейсы приведены в таблице 8.

Таблица 8 – Тест-кейсы алгоритма NNH-TW

Название теста	Проверяемое условие
TestOptimize_TwoNodesNoWindows	Граф из 2 узлов без временных ограничений; оба посещены ровно один раз
TestOptimize_ThreeNodesNearestNeighbor	3 узла; ближайший сосед выбирается корректно (порядок 0→1→2)
TestOptimize_TimeWindowEarliestFirst	Стартовый узел выбирается по наиболее раннему временному окну
TestOptimize_WaitBeforeWindow	Ожидание перед открытием окна (TotalWaitSec > 0)
TestOptimize_FallbackPass	Проход 2: нода выбирается без учета окна при отсутствии допустимых узлов
TestOptimize_AggregateStats	Корректность агрегатных показателей (расстояние, время, сервис)
TestOptimize_OneNode	Граф из одного узла; order = [0]
TestOptimize_StartTime0800	Начало работы в 08:00; оба узла посещены
TestOptimize_SymmetricMatrix	Симметричная матрица из 4 узлов; каждый посещен ровно один раз
TestOptimize_UnreachablePairs	«Недостижимые» пары (99 999 сек); все узлы посещены через Pass 2
TestOptimize_EmptyGraph	Пустой граф; order = [] без ошибок
TestFeasible_*(5 случаев)	Граничные значения функции feasible(): нет ограничений, в пределах окна, до открытия, после закрытия, на границе
TestOptimize_FixedStartNode	Фиксированная начальная точка: маршрут начинается с указанного узла
TestOptimize_FixedEndNode	Фиксированная конечная точка: маршрут завершается указанным узлом
TestOptimize_FixedStartAndEnd	Одновременное задание начальной и конечной точек
TestOptimize_PrecedenceConstraint	Ограничение предшествования: узел A посещается раньше узла B
TestOptimize_AllVisitedWithConstraints	Все узлы посещены при наличии комплексных ограничений
TestOptimize_LookAheadPreventsWindowMiss	Просмотр вперед предотвращает пропуск узла с жестким дедлайном
TestOptimize_TightDeadlineTiebreaker	Узел с более срочным дедлайном выбирается при равном времени завершения
TestOptimize_TimingStops	Корректность таймингов остановок

2.4.3 Тестирование бизнес-логики серверной части

Варианты использования тестируются с использованием репозитория заглушек, генерируемых пакетом `testify/mock`. Такой подход позволяет тестировать бизнес-логику в изоляции от базы данных.

Покрываются следующие компоненты:

- `auth/story` — регистрация, вход, выход, обновление токена;
- `task/story` — создание, редактирование, удаление задачи, проверка принадлежности задачи пользователю;
- `route/story` — сохранение оптимизированного маршрута, получение маршрута по идентификатору.

2.4.4 Тестирование клиентской части

Клиентская часть тестируется с использованием стека: `vitest` — тестовый раннер, `@testing-library/react` — тестирование компонентов, `msw` (Mock Service Worker) — перехват и подмена HTTP-запросов в тестах.

Ключевые тест-кейсы клиентской части:

- Логика обновления токена: при получении ответа 401 запускается ровно один запрос обновления (дедупликация через общий Promise); после успешного обновления исходный запрос повторяется;
- Утилита построения матрицы расстояний: корректное формирование $n \times (n - 1)$ запросов к мультимаршрутизатору Яндекс JS API.

2.4.5 Вывод

Реализованная стратегия тестирования охватывает все критические компоненты системы: алгоритм оптимизации (35 тест-кейсов, включая про-

смотр вперед и ограничения), бизнес-логику серверной части (тесты через заглушки) и клиентский API-клиент (тесты с MSW). Использование интерфейсов и внедрение зависимостей обеспечивает возможность полноценного тестирования бизнес-логики без обращения к реальной базе данных.

2.5 Интерфейс

На рисунках 4, 5, 6, 7 представлены примерные макеты разрабатываемого приложения:

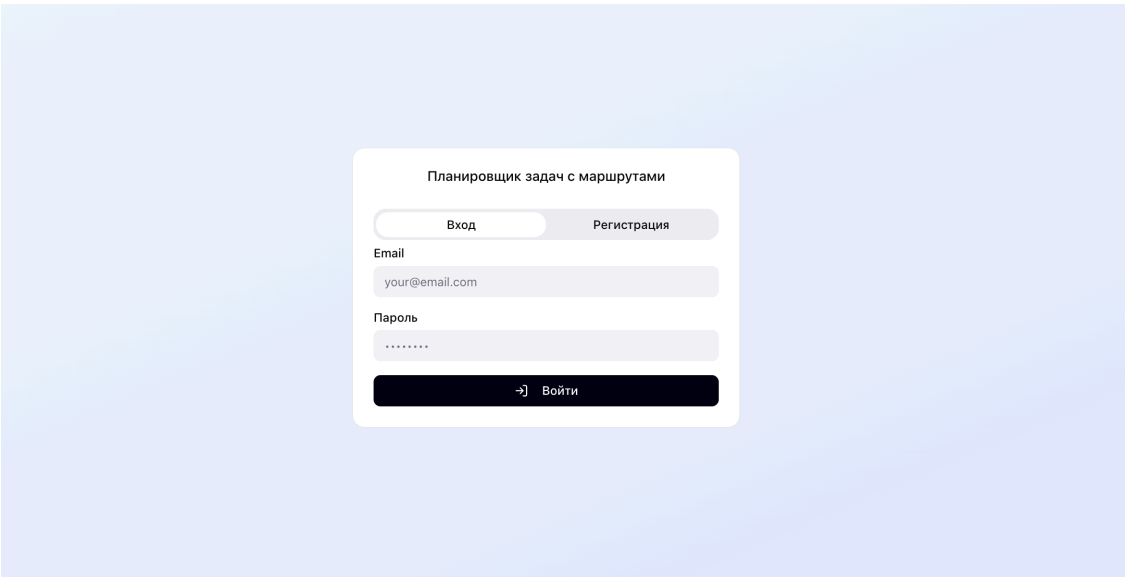


Рисунок 4 – Макет страницы с авторизацией

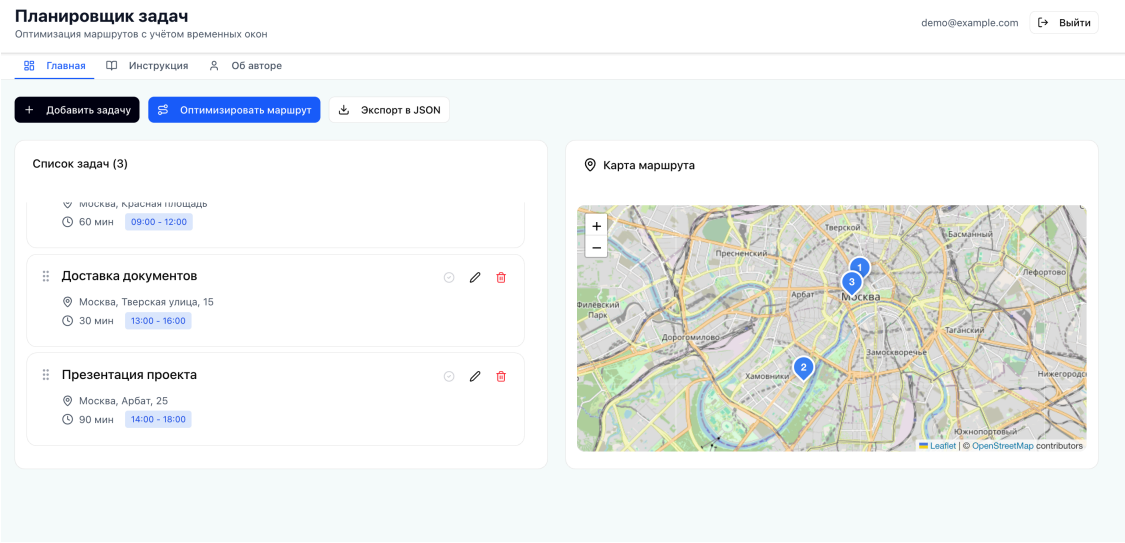


Рисунок 5 – Макет главной страницы приложения

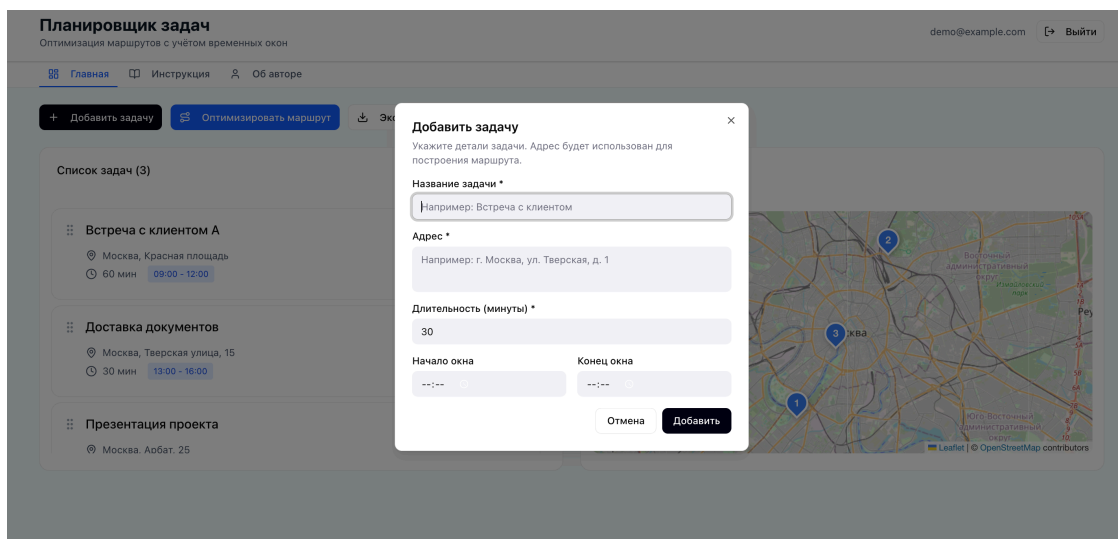


Рисунок 6 – Макет диалогового окна для создания новой задачи

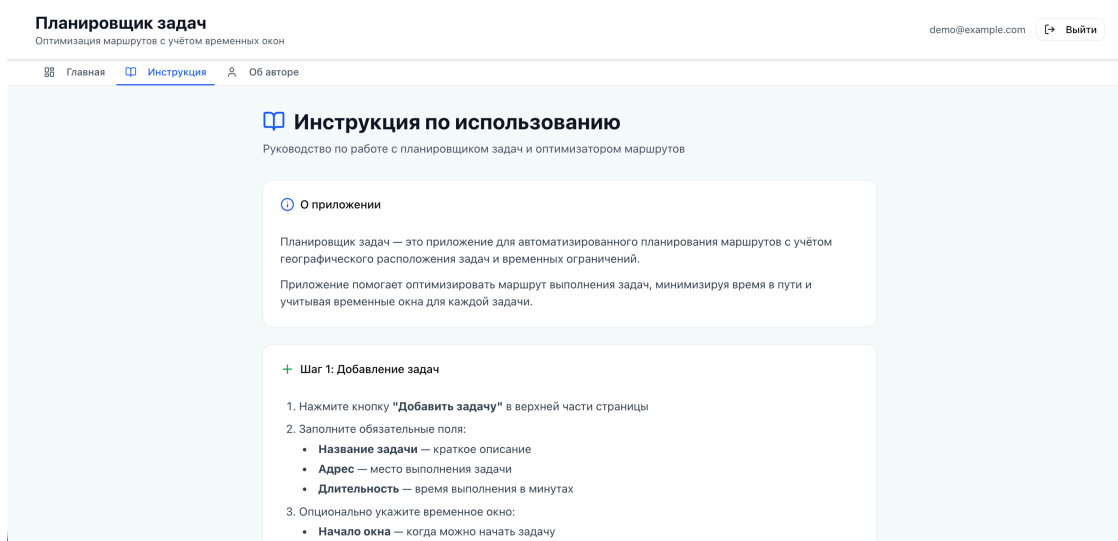


Рисунок 7 – Макет страницы с инструкций к приложению

2.5.1 Общая логика построения интерфейса

На главной странице (рисунок 5) список задач и карта маршрута размещены в пределах одного экрана, что позволяет немедленно оценивать влияние изменений в списке задач на итоговый маршрут без переключения между представлениями.

2.5.2 Организация рабочей области

Рабочая область разделена на две функциональные зоны:

— список задач;

— карта с визуализацией маршрута.

Список задач представлен в виде отдельных карточек с фиксированным набором полей, что обеспечивает визуальное разграничение элементов при одновременном отображении нескольких задач.

Карта размещена справа, что обеспечивает постоянную визуальную связь между порядком задач и их пространственным расположением. Пользователь может сопоставить маршрут с реальным расположением точек без дополнительных переходов между страницами.

2.5.3 Организация ввода данных

Добавление новой задачи реализовано через модальное окно (рисунок 6).

Поля формы сгруппированы логически: название, адрес, длительность, временное окно. Такая последовательность соответствует естественному порядку формулирования задачи.

2.5.4 Навигационная структура

Навигация организована через верхнюю панель (рисунок 7).

В интерфейсе предусмотрены следующие разделы:

- главная страница — работа с задачами и маршрутом;
- инструкция — описание принципа работы системы.

2.5.5 Импорт и экспорт задач

Для ускорения ввода большого количества задач реализован импорт из файлов CSV и Excel (XLSX). Диалог импорта отображает распознанные строки, выделяет ошибки валидации (отсутствие названия, некорректный формат времени) и позволяет пользователю подтвердить или отклонить каж-

дую строку перед сохранением. Импортированные задачи создаются пакетно через обработчик HTTP-запросов `POST /api/tasks/batch`.

Экспорт задач доступен в двух форматах: CSV (библиотека `raparparse`) и Excel (библиотека `xlsx`). Экспортируются все текущие задачи пользователя с полями: название, адрес, длительность, дата и время временного окна.

2.5.6 Поддержка пользовательского контроля

В интерфейсе реализована возможность изменения порядка задач методом перетаскивания.

Даже при наличии автоматической оптимизации пользователь сохраняет возможность ручного управления последовательностью выполнения. Это обеспечивает соответствие принципу пользовательского контроля: оператор может скорректировать автоматически сформированный порядок без повторного запуска алгоритма.

Дополнительно реализованы следующие элементы управления:

- **Выбор режима транспорта:** пользователь выбирает способ перемещения (автомобиль, пешком, общественный транспорт), что влияет на матрицу расстояний и визуализацию маршрута на карте. Для режима «общественный транспорт» расчёт времени приближается авто-режимом из-за ограничений API мультимаршрутизации, тогда как построение и отображение маршрута производится корректно для всех режимов;

- **Фиксация начальной и конечной точки:** через контекстное меню карточки задачи пользователь назначает точку старта или финиша маршрута;

- **Ограничения порядка:** пользователь задает пары «задача А до задачи В», которые алгоритм обязан соблюдать;

- **Обнаружение конфликтов:** интерфейс автоматически выявляет и подсвечивает задачи с некорректными или конфликтующими временными

окнами как до оптимизации (взаимное перекрытие), так и после (фактическое опоздание);

- **Отметка выполнения:** задача может быть помечена как выполненная;
- **Клонирование задач:** дублирование существующей задачи для быстрого создания аналогичной записи.

2.6 Техническая реализация

2.6.1 Состав и объем программного кода

Разработанное приложение состоит из двух независимых программных компонентов: серверной части на языке Go и клиентской части на языке TypeScript с использованием фреймворка React.

Серверная часть насчитывает 50 исходных файлов на языке Go, организованных в соответствии с принципами чистой архитектуры. Код распределен по следующим функциональным пакетам:

- `cmd/api` — точка входа в приложение (1 файл);
- `internal/config` — загрузка конфигурации из переменных окружения (1 файл);
- `internal/app` — сборка зависимостей (1 файл);
- `internal/common/ptrs` — общие вспомогательные функции (1 файл);
- `internal/domain` — бизнес-логика предметной области: сущности, интерфейсы репозиториев и варианты использования для модулей аутентификации, задач, маршрутов, пользователей и пакетного импорта задач (27 файлов, включая тесты);
- `internal/api/http` — HTTP-обработчики, промежуточное программное обеспечение, маршрутизатор и объекты передачи данных (12 файлов, включая тесты);

- `internal/platform/postgres` — реализации репозитория и подключение к базе данных (5 файлов);
- `internal/platform/distance` — провайдер матрицы расстояний с реализацией на основе OSRM и LRU-кэшем с TTL на базе библиотеки `hashicorp/golang-lru/v2` (3 файла);
- `migrations` — SQL-миграции схемы базы данных (5 миграций, 10 файлов).

Клиентская часть содержит 77 исходных файлов TypeScript и TSX. Из них около 30 файлов реализуют прикладную логику и компоненты страниц, остальные относятся к библиотеке пользовательского интерфейса на основе `shadcn/ui` и `Radix` (примитивы пользовательского интерфейса). Структура клиентской части включает:

- корневые компоненты приложения и конфигурация маршрутизации (3 файла);
- клиент программного интерфейса с логикой обновления токена (2 файла);
- контекст аутентификации и хук `useAuth()` (1 файл);
- типы TypeScript для доменных сущностей (3 файла);
- утилиты загрузки Яндекс JS API и построения матрицы расстояний (2 файла);
- компоненты страниц и функциональные компоненты (11 файлов);
- компоненты библиотеки пользовательского интерфейса (48 файлов).

Наиболее значимые фрагменты кода приведены в приложениях: реализация алгоритма оптимизации маршрута — в приложении Б.1, обработчики HTTP-запросов для маршрутов — в приложении Б.2, клиент API с логикой обновления токена — в приложении Б.3, утилита построения матрицы расстояний — в приложении Б.4.

2.6.2 Используемые библиотеки серверной части

В серверной части используются следующие сторонние библиотеки Go:

- `github.com/gin-gonic/gin` (v1.10.0) — HTTP-фреймворк для построения REST API; обеспечивает маршрутизацию запросов, группировку обработчиков, обработку ошибок и привязку JSON-тела запроса к структурам данных;

- `github.com/gin-contrib/cors` (v1.7.2) — промежуточное программное обеспечение для настройки политики CORS (Cross-Origin Resource Sharing);

- `github.com/golang-jwt/jwt/v5` (v5.2.1) — генерация и верификация JSON Web Token для реализации механизма аутентификации без сохранения состояния на сервере;

- `github.com/jackc/pgx/v5` (v5.6.0) — низкоуровневый драйвер PostgreSQL для Go с поддержкой пула соединений; используется вместо стандартного `database/sql` ввиду более высокой производительности и поддержки специфичных типов PostgreSQL;

- `github.com/google/uuid` (v1.6.0) — генерация уникальных идентификаторов UUID версии 4 для первичных ключей всех таблиц;

- `golang.org/x/crypto` (v0.27.0) — функции криптографии, в частности `bcrypt` для хэширования паролей пользователей и токенов обновления;

- `github.com/stretchr/testify` (v1.9.0) — набор вспомогательных функций для написания тестов: утверждения `assert/require` и создание заглушек через пакет `mock`.

Для управления миграциями схемы базы данных используется инструмент `migrate/migrate` (v4.17.1), запускаемый автоматически при старте Docker-контейнера.

2.6.3 Используемые библиотеки клиентской части

В клиентской части задействованы следующие основные библиотеки:

- `react` и `react-dom` (v18.3.1) — базовый фреймворк для построения компонентного пользовательского интерфейса;
- `react-router` (v7.13.0) — декларативная клиентская маршрутизация; используется для разделения приложения на страницы без перезагрузки браузера;
- `vite` (v6.3.5) с плагином `@vitejs/plugin-react` — инструмент сборки; обеспечивает быстрое обновление модулей в режиме разработки (HMR) и оптимизированную сборку для продакшена;
- `tailwindcss` (v4) — утилитарный CSS-фреймворк; используется для стилизации компонентов без написания отдельных файлов стилей;
- пакеты `@radix-ui/*` (более 20 пакетов) — безголовые примитивы доступного пользовательского интерфейса (диалоги, всплывающие подсказки, вкладки, аккордеон и др.); служат основой для компонентов `shadcn/ui`;
- `react-hook-form` (v7.55.0) — управление состоянием форм с минимальным количеством перерисовок; применяется в форме создания задачи;
- `react-dnd` и `react-dnd-html5-backend` — реализация перетаскивания элементов (`drag-and-drop`) для ручного упорядочивания задач;
- `sonner` (v2.0.3) — компонент всплывающих уведомлений (`toast`);
- `lucide-react` — библиотека SVG-иконок;
- `date-fns` — утилиты для форматирования дат и работы со временем;
- `papaparse` (v5.5) — парсинг и генерация файлов CSV; используется для импорта и экспорта задач;
- `xlsx` (v0.18) — чтение и запись файлов Excel (XLSX); используется для импорта и экспорта задач.

Для тестирования клиентской части используется стек: `vitest` (v3.2.4) как тестовый раннер, `@testing-library/react` для тестиро-

вания компонентов, @testing-library/user-event для симуляции пользовательских действий, а также msw (Mock Service Worker) для перехвата и подмены HTTP-запросов в тестах.

Картографический функционал реализован на основе Яндекс JavaScript API и HTTP Геокодер версии 2.1 [22], загружаемого динамически через `<script>`-тег при первом обращении к карте. Данный API предоставляет геокодер, механизм мультимаршрутизации для построения матрицы расстояний и инструменты визуализации (маркеры, полилинии, управление видимой областью карты).

2.6.4 Требования к техническим средствам

Для развертывания и эксплуатации приложения предъявляются следующие требования к техническим средствам.

Для серверной части:

- процессор: одноядерный с тактовой частотой не ниже 1 ГГц (рекомендуется двухъядерный и более);
- оперативная память: не менее 512 МБ (рекомендуется 1 ГБ и более);
- дисковое пространство: не менее 1 ГБ для хранения образов Docker и данных PostgreSQL;
- сетевой интерфейс: соединение с интернетом для обращения к внешним сервисам (Яндекс Карты, OSRM).

Для клиентской части (рабочее место пользователя):

- устройство с современным веб-браузером и поддержкой JavaScript;
- соединение с интернетом для загрузки Яндекс Карт и обращения к API.

2.6.5 Требования к программному обеспечению

Для развертывания серверной части с использованием Docker необходимо наличие:

— операционной системы Linux, macOS или Windows с поддержкой контейнеризации;

— Docker Engine версии 24.0 и выше;

— Docker Compose версии 2.0 и выше.

При запуске серверной части без Docker дополнительно требуются:

— Go версии 1.23 и выше;

— PostgreSQL версии 16 с расширением PostGIS версии 3.4;

— утилита `migrate` для применения миграций.

Для сборки и запуска клиентской части в режиме разработки необходимы:

— Node.js версии 18 и выше;

— менеджер пакетов `pnpm` версии 8 и выше (либо `npm` версии 9 и выше).

На стороне пользователя поддерживаются следующие браузеры: Google Chrome версии 90 и выше, Mozilla Firefox версии 88 и выше, Apple Safari версии 14 и выше, Microsoft Edge версии 90 и выше.

2.6.6 Вывод

Серверная часть содержит 50 файлов на языке Go, клиентская — 77 файлов на языке TypeScript.

2.7 Руководство пользователя и администратора

2.7.1 Общие сведения о развертывании

Приложение состоит из двух независимо запускаемых компонентов: серверной части (Go API + PostgreSQL) и клиентской части (React SPA). Серверная часть распространяется в виде Docker-образов и предназначена для запуска на сервере или локальной машине. Клиентская часть запускается в

веб-браузере и не требует установки дополнительного программного обеспечения на стороне пользователя.

Поддерживаются следующие платформы для развертывания серверной части: Linux (рекомендуется), macOS и Windows с установленным Docker Engine.

2.7.2 Установка и настройка серверной части

2.7.2.1 Развертывание с использованием Docker Compose

Рекомендуемым способом развертывания является использование Docker Compose, который автоматически поднимает все необходимые сервисы: базу данных PostgreSQL с расширением PostGIS, службу применения миграций и сам API-сервер.

Последовательность установки:

а) Убедиться в наличии установленных Docker Engine (версии 24.0 и выше) и Docker Compose (версии 2.0 и выше).

б) Перейти в директорию `backend/`.

в) При необходимости изменить переменные среды непосредственно в файле `docker-compose.yml` или передать их через файл `.env`.

г) Выполнить команду:

```
docker compose up --build
```

После выполнения указанных шагов Docker Compose последовательно: запустит контейнер базы данных PostgreSQL, дождется готовности СУБД (проверка через `pg_isready`), применит SQL-миграции через утилиту `migrate`, запустит API-сервер на порту 8080.

2.7.2.2 Параметры конфигурации

Конфигурация серверной части осуществляется через переменные окружения. Перечень параметров приведен в таблице 9.

Таблица 9 – Переменные окружения серверной части

Переменная	Значение по умолчанию	Описание
APP_HTTP_ADDR	:8080	Адрес и порт HTTP-сервера
APP_DB_DSN	—	Строка подключения к PostgreSQL
APP_JWT_SECRET	—	Секретный ключ для подписи JWT
APP_ACCESS_TTL_MIN	15	Время жизни access-токена (минуты)
APP_REFRESH_TTL_DAYS	30	Время жизни refresh-токена (дни)
APP_CORS_ORIGINS	http://localhost:5173	Допустимые источники CORS
APP_OSRM_BASE_URL	http://router.project-osrm.org	Базовый URL OSRM-сервиса

Переменные APP_DB_DSN и APP_JWT_SECRET являются обязательными при запуске без Docker Compose. Для продуктивной эксплуатации значение APP_JWT_SECRET должно быть заменено на криптографически стойкую случайную строку длиной не менее 32 символов.

2.7.2.3 Развертывание без Docker

При запуске серверной части без контейнеризации необходимо:

- Установить Go версии 1.23 и выше, PostgreSQL версии 16 с расширением PostGIS версии 3.4, а также утилиту migrate.
- Создать базу данных и пользователя PostgreSQL с правами на нее.
- Скопировать файл конфигурации:

```
cp .env.example .env
```


и заполнить параметры APP_DB_DSN и APP_JWT_SECRET.

г) Применить миграции вручную:

```
migrate -path ./migrations -database "<DSN>" up
```

д) Запустить сервер:

```
make run
```

2.7.3 Установка и настройка клиентской части

2.7.3.1 Режим разработки

Для запуска клиентского приложения в режиме разработки необходимо:

а) Убедиться в наличии Node.js версии 18 и выше, менеджера пакетов npm версии 8 и выше (либо npm версии 9 и выше).

б) Перейти в директорию frontend/Task Planning App/.

в) Создать файл .env.local и указать ключ Яндекс JS API:

```
VITE_YANDEX_GEOCODER_KEY=<ключ>
```

Ключ необходимо получить на портале разработчиков Яндекса, подключив сервис «JavaScript API и HTTP Геокодер». Активация ключа занимает до 15 минут после регистрации.

г) Установить зависимости и запустить сервер разработки:

```
pnpm install
```

```
pnpm dev
```

Приложение будет доступно по адресу `http://localhost:5173`. Все запросы к адресам вида `/api/*` автоматически проксируются на серверную часть по адресу `http://localhost:8080` через встроенный механизм Vite.

2.7.3.2 Сборка для продуктивной эксплуатации

Для получения оптимизированной статической сборки выполняется команда:

```
pnpm build
```

Собранные файлы размещаются в директории `dist/` и могут быть опубликованы на любом веб-сервере (Nginx, Apache, Caddy и других). При этом необходимо настроить перенаправление всех запросов на файл `index.html` для корректной работы клиентской маршрутизации, а также проксирование запросов к API на серверную часть.

2.7.4 Перечень возможных затруднений и способы их устранения

В таблице 10 приведены типичные проблемы, возникающие при установке и эксплуатации приложения, и способы их устранения.

Таблица 10 – Типичные проблемы и способы их устранения

Проблема	Способ устранения
API-сервер не запускается: ошибка подключения к базе данных	Убедиться, что контейнер db запущен и прошел проверку healthcheck; проверить корректность строки APP_DB_DSN
Карта не отображается в браузере	Проверить наличие и корректность ключа VITE_YANDEX_GEOCODER_KEY в файле .env.local; убедиться в доступности серверов Яндекс JS API
Геокодирование не возвращает результатов	Проверить лимиты использования API (до 1 000 запросов в сутки на бесплатном тарифе); при превышении — подождать до следующего дня или перейти на платный тариф
Порт 8080 или 5432 уже занят	Изменить проброс портов в docker-compose.yml (например, "18080:8080") и соответственно обновить значение APP_CORS_ORIGINS
Изменения в .env.local не применяются	Перезапустить сервер разработки Vite: он не выполняет горячую перезагрузку переменных окружения
Ошибка при применении миграций	Проверить, что база данных создана и пользователь обладает правами на создание таблиц; при частичном применении выполнить откат командой migrate ... down и повторно up

2.7.5 Краткое руководство пользователя

Для начала работы с приложением пользователю необходимо:

- а) Открыть приложение в браузере и зарегистрироваться, указав адрес электронной почты и пароль, либо войти в существующую учетную запись.
- б) На главной странице нажать кнопку добавления задачи. В открывшемся модальном окне ввести название задачи, адрес выполнения, продолжительность и временное окно (при необходимости).

в) После добавления нескольких задач нажать кнопку «Оптимизировать маршрут». Приложение выполнит геокодирование адресов, получит матрицу расстояний и вычислит оптимальный порядок задач.

г) Результат отображается одновременно в списке задач (в оптимальном порядке) и на карте (в виде маршрутной линии с пронумерованными метками).

д) При необходимости пользователь может вручную изменить порядок задач методом перетаскивания карточек.

е) Для удаления или редактирования задачи следует воспользоваться соответствующими элементами управления на карточке задачи.

2.7.6 Руководство администратора

Администратор системы несет ответственность за развертывание, поддержание работоспособности и обновление серверной части приложения.

Резервное копирование. Данные PostgreSQL хранятся в именованном томе Docker `db_data`. Для создания резервной копии следует использовать утилиту `pg_dump`:

```
docker exec <container_db> pg_dump -U planner planner > bac
```

Восстановление из резервной копии:

```
docker exec -i <container_db> psql -U planner planner < bac
```

Обновление приложения. При выходе новой версии необходимо пересобрать образы командой `docker compose up --build`, которая автоматически применит новые миграции базы данных перед запуском сервера.

Мониторинг. Состояние контейнеров отслеживается командой `docker compose ps`, журналы приложения — командой `docker compose logs api`.

Обеспечение безопасности. В продуктивной среде необходимо: заменить все пароли по умолчанию в `docker-compose.yml`, установить значение `APP_JWT_SECRET` равным криптографически стойкой случайной строке, ограничить доступ к портам 5432 и 8080 на уровне межсетевого экрана, предоставив внешний доступ только к порту 8080 (или разместив приложение за обратным прокси-сервером с поддержкой TLS).

2.7.7 Вывод

Приложение поддерживает два способа развертывания серверной части: с использованием Docker Compose (рекомендуется) и без контейнеризации. Docker Compose обеспечивает автоматическое применение миграций и изоляцию зависимостей, что упрощает как первоначальную установку, так и дальнейшее обслуживание системы.

Руководство пользователя описывает полный цикл работы с приложением, а руководство администратора охватывает процедуры резервного копирования, обновления и обеспечения безопасности, необходимые для продуктивной эксплуатации.

2.8 Выводы

В настоящей главе спроектирована и реализована программная система автоматизированного планирования маршрутов с учетом временных окон. Разработана модель данных, опирающаяся на расширение PostGIS и поддерживающая хранение задач, маршрутов, временных окон и пользовательских ограничений с обеспечением целостности и масштабируемости.

В качестве алгоритмической основы выбрана и подробно описана эвристика ближайшего соседа с временными окнами и одношаговым просмотром вперед (NNH-TW). Для алгоритма выполнен расчет вычислительной сложности $O(n^3)$ и приведена пошаговая трассировка работы на иллюстративном

примере. Реализована поддержка дополнительных пользовательских ограничений: фиксированных начальной и конечной точек маршрута, а также отношений предшествования между задачами.

Серверная часть приложения построена в соответствии с принципами чистой архитектуры с явным разделением слоев домена, прецедентов и инфраструктуры, что обеспечивает независимость бизнес-логики от внешних библиотек и упрощает сопровождение. Клиентская часть реализована как одностраничное приложение с использованием React и TypeScript, поддерживает визуализацию маршрута на карте, импорт и экспорт списка задач, а также взаимодействие с серверной частью через защищенный программный интерфейс REST.

Описаны процедуры развертывания приложения с использованием Docker Compose и без контейнеризации, подготовлены руководства пользователя и администратора, охватывающие установку, настройку, резервное копирование и обеспечение безопасности продуктивной среды. Полученные результаты показывают, что разработанная архитектура и выбранные алгоритмические решения позволяют создать работоспособную программную систему, удовлетворяющую функциональным и нефункциональным требованиям, сформулированным в первой главе.

3 ВНЕДРЕНИЕ И ЭКСПЛУАТАЦИЯ

3.1 Анализ внедрения продукта

В первой главе сформулированы цель разработки и перечень функциональных и нефункциональных требований к системе. В данном разделе проводится сопоставление этих требований с результатами реализации.

Все сформулированные функциональные требования реализованы в полном объеме. Полное соответствие реализации перечню требований приведено в таблице 11; ниже перечислены технические решения, которые не очевидны из её строк.

Матрица расстояний формируется двумя путями: в основном сценарии клиент рассчитывает её через мультимаршрутизатор Яндекс JavaScript API и HTTP Геокодер [22] и передаёт серверу в теле запроса на оптимизацию, что согласует данные оптимизации с маршрутом, отображаемым на карте; при отсутствии клиентской матрицы сервер строит её через свободный сервис OSRM [20, 21]. Полученные значения кэшируются на уровне приложения в оперативной памяти для повторного использования между запросами.

Алгоритм оптимизации помимо временных окон поддерживает фиксированные начальную и конечную точки маршрута, а также попарные ограничения порядка посещения, рассчитывая для каждой остановки время прибытия, начала и окончания обслуживания.

Нефункциональные требования также выполнены. Серверная часть реализована на языке Go с использованием HTTP-фреймворка Gin. Go компилируется в нативный исполняемый файл, а модель конкурентности на основе горутин позволяет обслуживать множество одновременных соединений без выделения отдельного потока операционной системы на каждый запрос [14]. Требование по времени ответа обеспечивается за счет использования пула соединений драйвера `pgx/v5`, кэширования матри-

цы расстояний в оперативной памяти приложения и вычисления матрицы на клиентской стороне через мультимаршрутизатор Яндекс JavaScript API и HTTP Геокодер [22], что снижает нагрузку на сервер при оптимизации. Устойчивость к повторным запросам к внешним API обеспечивается кэшем в памяти приложения (`CachedProvider` на основе библиотеки `hashicorp/golang-lru/v2` [47], пакет `expirable`): при наличии ранее вычисленного значения между парой точек обращение к OSRM не производится. Контейнеризация на основе `Docker Compose` обеспечивает воспроизводимость развертывания и независимость от конфигурации целевого сервера. Поддержка современных браузеров обеспечивается выбором клиентского стека (`React 18 + Vite 6 + TypeScript`): согласно официальной документации, `React 18` поддерживает все браузеры, совместимые с `ES2015` [5], а `Vite 6` нацеливается на браузеры с поддержкой нативных ES-модулей [26] (`Chrome 89+`, `Firefox 89+`, `Safari 15+`, `Edge 89+`).

Сравнение запланированных требований с результатами реализации представлено в таблице 11.

Таблица 11 – Соответствие реализованной системы поставленным требованиям

Требование	Результат
Создание, редактирование и удаление задач	Реализовано
Хранение географических координат	Реализовано
Получение матрицы расстояний через внешние API	Реализовано (Яндекс мультимаршрутизатор на клиенте, OSRM как резервный, кэш в памяти приложения)
Вычисление оптимальной последовательности с учетом временных окон	Реализовано
Расчет времени прибытия, ожидания и завершения	Реализовано
Визуализация маршрута на карте	Реализовано (Яндекс Карты)
Экспорт задач в форматах CSV и XLSX	Реализовано
Импорт задач из CSV и XLSX	Реализовано
Дополнительные ограничения маршрута (начальная/конечная точка, порядок)	Реализовано
Выбор режима транспорта	Реализовано (авто, пешком, общественный транспорт; в последнем случае расчёт времени приближается авто-режимом, визуализация выполняется корректно)
Обнаружение конфликтов временных окон	Реализовано
Время ответа сервера не более 2 секунд	Выполнено
Поддержка современных браузеров	Выполнено
Устойчивость к повторным запросам API за счет кэширования	Выполнено
Масштабируемость до 100 одновременных пользователей	Подтверждено нагрузочным тестированием: при 100 VU p95 = 1370 мс < 2000 мс, доля ошибок — 0 %

Выполнение всех функциональных и нефункциональных требований подтверждается таблицей 11.

3.2 Аспекты отказоустойчивости, надежности и производительности

3.2.1 Устойчивость к отказам внешних зависимостей

Одним из ключевых рисков систем, интегрирующих внешние сервисы, является деградация или временная недоступность стороннего API. В разработанном приложении этот риск снижается на двух уровнях.

На уровне приложения реализован кэш значений расстояния и времени перемещения между парами географических точек (`CachedProvider` над интерфейсом `distance.Provider`). В качестве хранилища используется библиотека `hashicorp/golang-lru/v2` (пакет `expirable`) [47] — потокобезопасный LRU-кэш с ограниченным размером и временем жизни записи, что исключает неконтролируемый рост занимаемой памяти. При наличии записи в кэше сервер не выполняет повторного обращения к OSRM. Это снижает зависимость от доступности внешнего маршрутизатора и уменьшает суммарную задержку при повторных оптимизациях маршрутов с теми же точками.

На уровне пользовательского интерфейса загрузка Яндекс JS API реализована через утилиту `loadYandexMaps()`, которая инжектирует `<script>`-тег динамически при первом обращении и возвращает один и тот же `Promise` при последующих вызовах (одиночный экземпляр). Такой подход предотвращает дублирующую загрузку скрипта при одновременном обращении нескольких компонентов и исключает состояние гонки при инициализации API.

3.2.2 Надежность аутентификации

Аутентификация реализована по схеме JWT [15] с двумя токенами: краткосрочным `access`-токеном (время жизни — 15 минут) и долго-

срочным `refresh`-токеном (время жизни — 30 дней). Применение пары `access/refresh`-токенов соответствует общему подходу к делегированной авторизации, описанному в [34]. Токены обновления хранятся в базе данных в виде хэша SHA-256, что исключает их компрометацию при несанкционированном доступе к СУБД; данная мера согласуется с практическими рекомендациями по защите сервисов, использующих JWT [35]. Пароли пользователей хэшируются алгоритмом `bcrypt` [16] с адаптивным количеством раундов.

На клиентской стороне реализован механизм автоматического обновления токена при получении ответа с кодом 401. Корректность этого механизма проверяется модульными тестами API-клиента с перехватом HTTP-запросов через `Mock Service Worker (MSW)`. Для исключения множественных одновременных запросов на обновление применяется дедупликация: при первом истечении токена запускается единственный запрос обновления, остальные запросы ожидают его завершения через общий `Promise`. После успешного обновления исходный запрос повторяется автоматически — пользователь не замечает прерывания сессии.

Сессия пользователя сохраняется в `localStorage` под ключом `planner.auth.session`. При закрытии и повторном открытии браузера приложение автоматически восстанавливает состояние аутентификации без повторного ввода учетных данных.

Диаграммы последовательности, отражающие три сценария взаимодействия компонентов системы аутентификации, представлены на рисунках 8, 9 и 10.

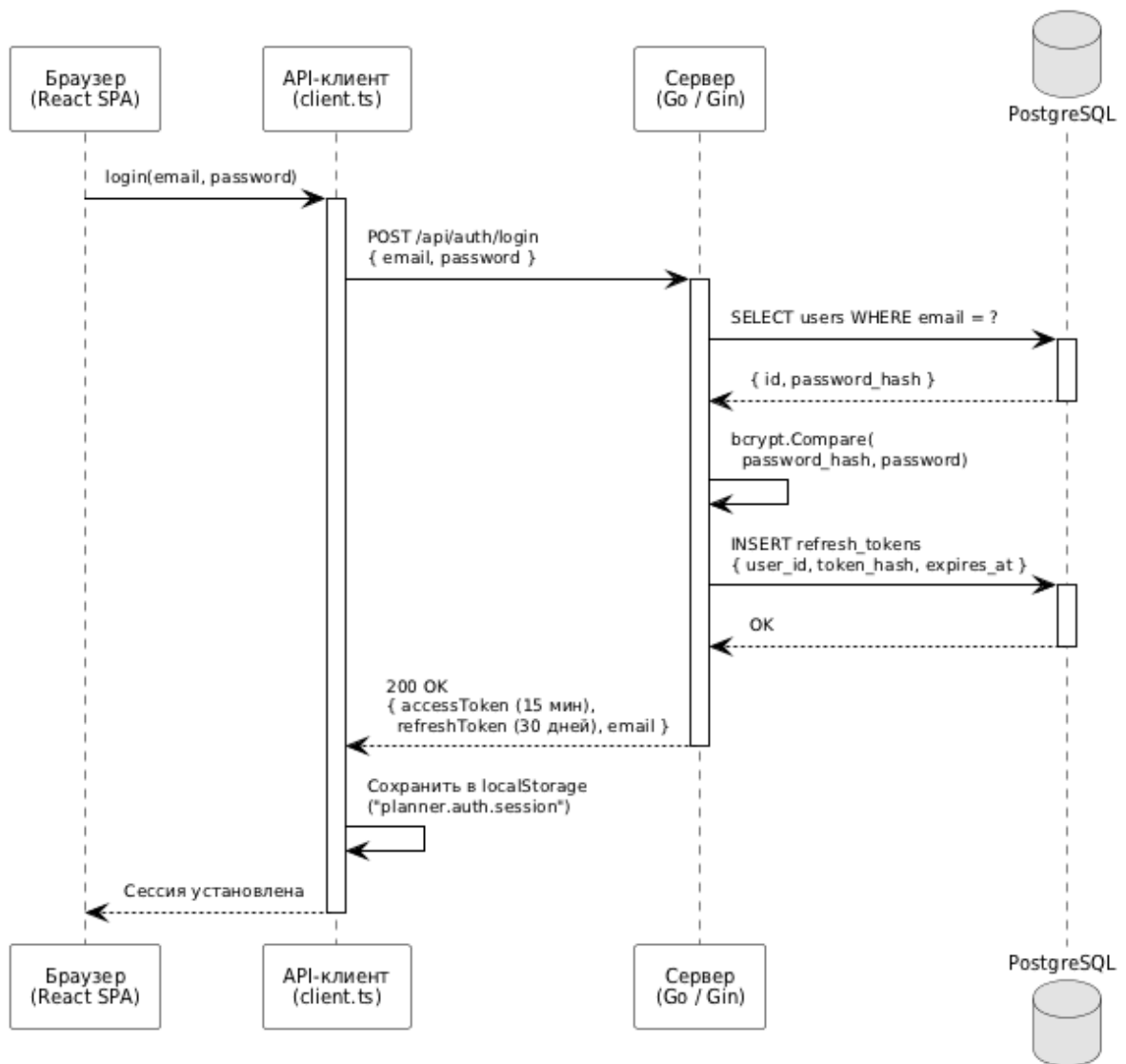


Рисунок 8 – Диаграмма последовательности: сценарий входа в систему

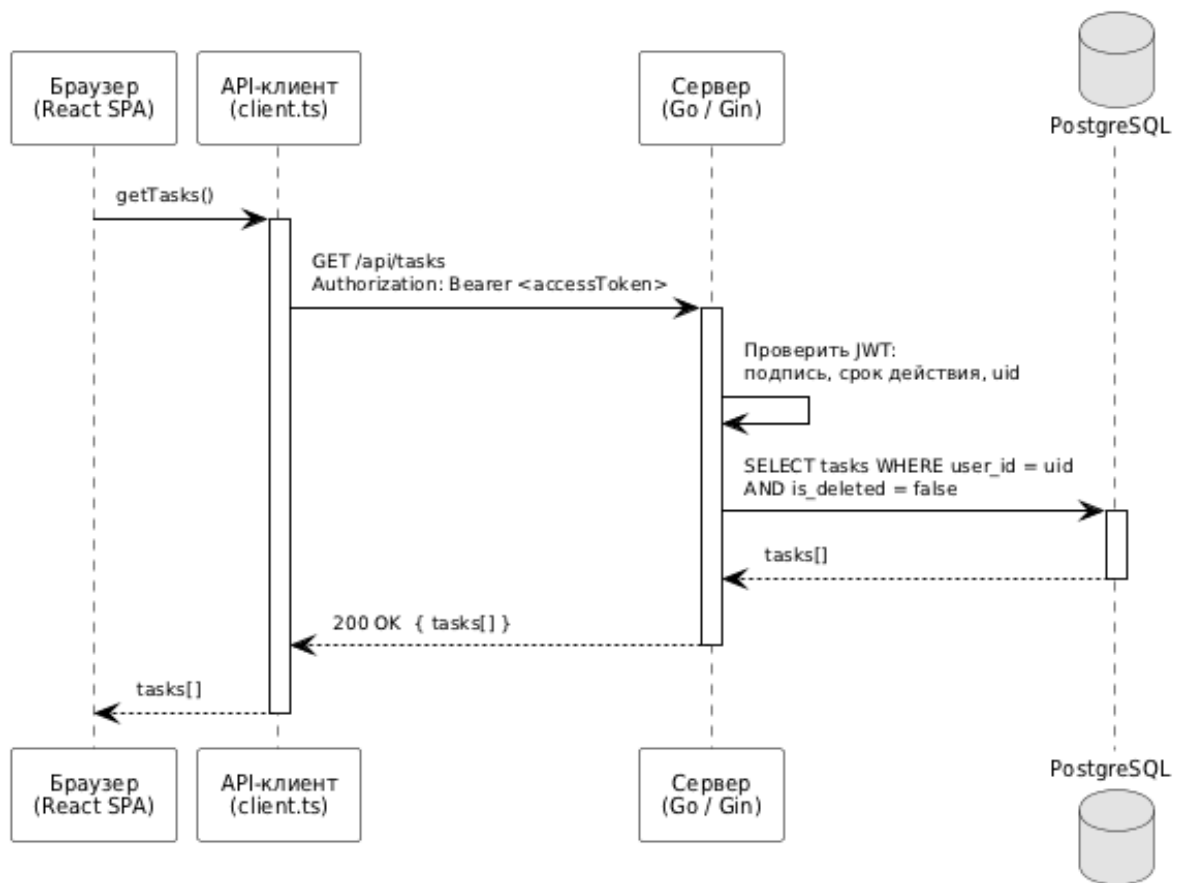


Рисунок 9 – Диаграмма последовательности: запрос с валидным токеном

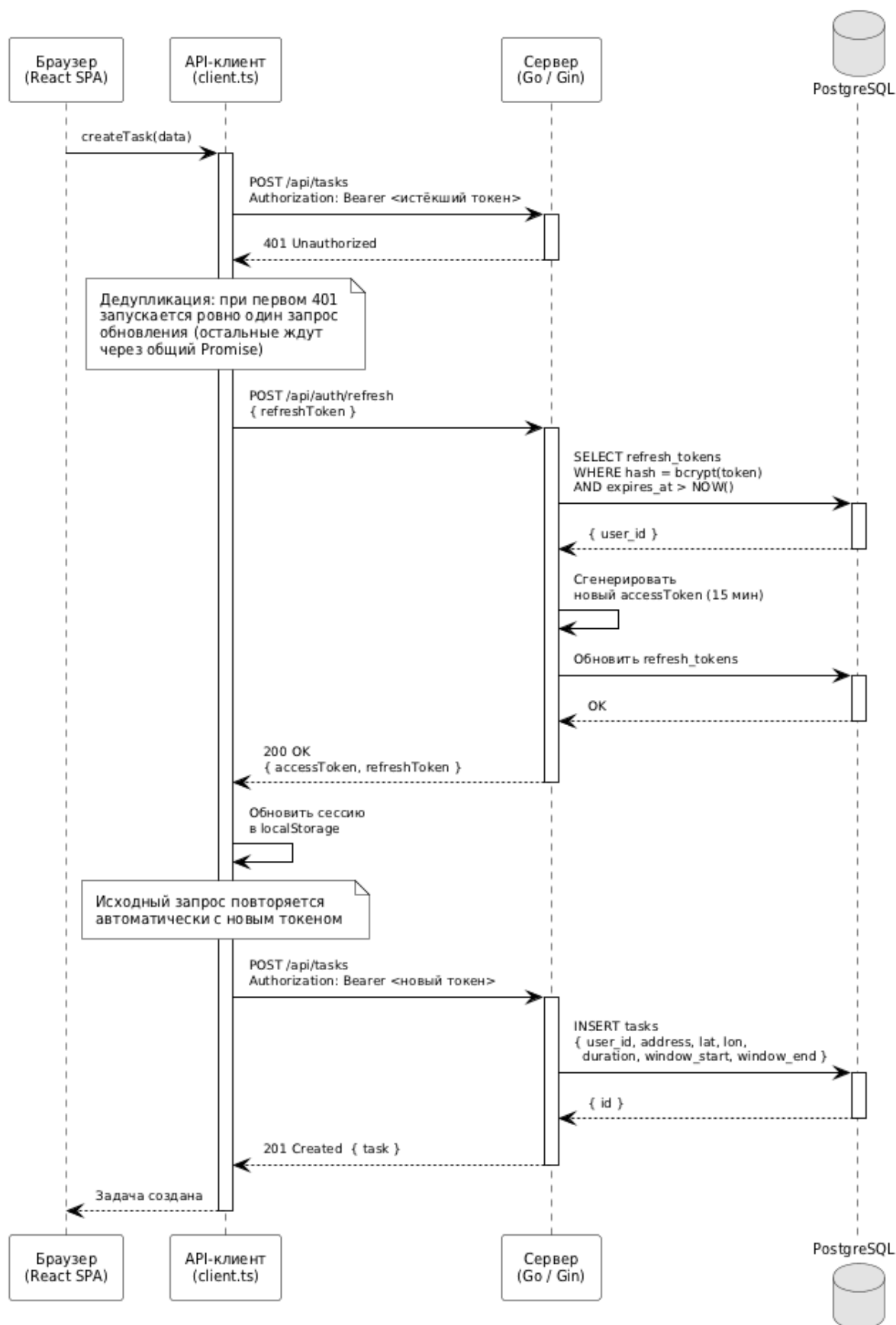


Рисунок 10 – Диаграмма последовательности:
автоматическое обновление токена при ответе 401

3.2.3 Надежность инфраструктуры

При разворачивании с использованием Docker Compose [6] зависимость между контейнерами управляется через механизм `healthcheck`: сервис применения миграций и API-сервер запускаются только после того, как контейнер базы данных PostgreSQL подтвердит готовность к приему соединений командой `pg_isready`. Это исключает ситуацию, при которой API-сервер стартует до завершения инициализации СУБД.

Схема базы данных управляется инструментом `migrate/migrate`, запускаемым автоматически при каждом старте стека. Применение миграций является идемпотентным: повторный запуск `migrate up` при уже актуальной схеме не вносит изменений. Это позволяет безопасно перезапускать стек без ручного контроля состояния миграций.

3.2.4 Производительность

Производительность серверной части обеспечивается несколькими механизмами. Язык Go компилируется в нативный исполняемый файл, что устраняет накладные расходы интерпретируемых сред. Драйвер `pgx/v5` использует пул соединений к PostgreSQL, исключая создание нового соединения на каждый HTTP-запрос. HTTP-фреймворк Gin применяет `httprouter` с маршрутизацией на дереве-префикса (`radix tree`), обеспечивая постоянное время поиска маршрута вне зависимости от числа зарегистрированных обработчиков HTTP-запросов.

Вычисления матрицы расстояний при оптимизации маршрута выполняются на стороне клиента посредством Яндекс JS API, что разгружает сервер от вычислительно емких операций и позволяет обслуживать большее число одновременных пользователей без изменения аппаратной конфигурации.

Требование нефункциональных характеристик об устойчивости к ограничениям внешних API выполнено за счет кэша расстояний в памяти приложения: при повторных оптимизациях маршрутов с теми же парами точек обращения к OSRM из серверной части не производятся.

3.2.5 Производительность алгоритма оптимизации

Для оценки производительности алгоритма NNH-TW использован встроенный инструмент бенчмаркинга языка Go (`testing.Benchmark`). Измерения проводились на рабочей станции с процессором Apple M1 (8 ядер) и 16 ГБ оперативной памяти под управлением операционной системы macOS 15. Для каждого значения n выполнялось не менее 50 000 итераций; в таблице приведено среднее время одного вызова. Результаты представлены в таблице 12.

Таблица 12 – Производительность алгоритма NNH-TW в зависимости от числа задач

Число задач (n)	Запросов к API матрицы ($n(n - 1)$)	Время NNH-TW (Go)	Узкое место
5	20	0,36 мкс	HTTP-запросы к мультимаршрутизатору Яндекс JS API
10	90	1,12 мкс	HTTP-запросы к мультимаршрутизатору Яндекс JS API
20	380	3,66 мкс	HTTP-запросы к мультимаршрутизатору Яндекс JS API
30	870	6,96 мкс	HTTP-запросы к мультимаршрутизатору Яндекс JS API
50	2450	24,54 мкс	HTTP-запросы к мультимаршрутизатору Яндекс JS API

Из таблицы 12 следует, что узким местом системы является не сам алгоритм оптимизации (сложность $O(n^3)$, выполнение при $n \leq 30$ за единицы-десятки микросекунд), а получение матрицы расстояний от внешнего API. При $n = 20$ задачах число параллельных запросов к Яндекс JS API составляет $20 \times 19 = 380$, однако эти запросы выполняются параллельно на стороне клиента через `Promise.all`, что сводит суммарную задержку к времени наиболее медленного из них; доминирующая задержка определяется сетевыми условиями и временем ответа внешнего API.

Кэш матрицы расстояний в памяти приложения снижает нагрузку на внешний API при повторных оптимизациях: если пара точек уже была обработана ранее, расстояние возвращается из кэша без обращения к OSRM.

3.2.6 Нагрузочное тестирование

Для проверки нефункционального требования о масштабируемости до 100 одновременных пользователей проведено нагрузочное тестирование REST API с использованием инструмента k6 [41]; Тест воспроизводит типовой сценарий работы пользователя: аутентификация, получение списка задач (`GET /api/tasks`) и создание новой задачи (`POST /api/tasks`). Каждый сценарий выполнялся в течение 60 секунд при фиксированном числе виртуальных пользователей (VU). Тестовый стенд: ноутбук с процессором Apple M2 (8 ядер), 16 ГБ оперативной памяти, серверный стек запущен локально через Docker Compose. Критерии успешности: 95-й перцентиль времени ответа менее 2000 мс, доля ошибочных HTTP-ответов менее 1 %.

Результаты нагрузочного тестирования приведены в таблице 13.

Таблица 13 – Результаты нагрузочного тестирования REST API

VU	Запросов	avg, мс	p90, мс	p95, мс	Ошибки
10	1500	69	133	159	0 %
50	5484	219	475	577	0 %
100	5799	720	1250	1370	0 %

Все три сценария выдержали установленные критерии успешности: ни один запрос не завершился ошибкой, а 95-й перцентиль времени ответа при 100 одновременных пользователей составил 1370 мс, что не превышает допустимого порога в 2000 мс. Таким образом, нефункциональное требование масштабируемости подтверждено экспериментально.

3.2.7 Вывод

Нагрузочное тестирование подтвердило выполнение требования масштабируемости: REST API выдерживает 100 одновременных пользователей без ошибок и с временем ответа (p95) в пределах установленного норматива.

3.3 Внедрение и сопровождение программного продукта

3.3.1 Модель распространения

Разработанное приложение является веб-приложением с клиент-серверной архитектурой и не требует установки специализированного программного обеспечения на рабочее место конечного пользователя. Пользователь взаимодействует с системой через стандартный веб-браузер.

Серверная часть распространяется в виде исходного кода и `Dockerfile`, что позволяет развернуть приложение на любой платформе с поддержкой Docker (Linux, macOS, Windows). Клиентская часть собирается в набор статических файлов (HTML, CSS, JavaScript) командой `pnpm build` и публикуется на любом веб-сервере (Nginx, Apache, Caddy и др.) без дополнительных зависимостей времени исполнения.

3.3.2 Процедура первоначального развертывания

Рекомендуемым способом развертывания является Docker Compose. Подробные инструкции по установке, конфигурированию и типичным затруднениям приведены в разделе «Руководство пользователя и администратора» второй главы. Перед переходом в продуктивную эксплуатацию необходимо обязательно изменить учетные данные СУБД, установить криптографически стойкое значение переменной `APP_JWT_SECRET` и задать корректный адрес клиентской части в переменной `APP_CORS_ORIGINS`.

3.3.3 Сопровождение и обновление

Обновление, резервное копирование и мониторинг выполняются стандартными средствами Docker. Процедуры обновления (команда `docker compose up --build`), создания резервных копий (`pg_dump`) и контроля состояния контейнеров подробно описаны в руководстве администратора второй главы.

3.3.4 Ограничения и направления развития

В текущей реализации хранение географических координат задач в базе данных осуществляется в соответствии с условиями бесплатного тарифа Яндекс JS API, которые допускают это для учебных и некоммерческих проектов. Для продуктивного использования в коммерческой среде потребуется переход на коммерческую лицензию API.

Суточный лимит бесплатного тарифа составляет 1 000 запросов к геокодеру и 25 000 запросов к JavaScript API. При превышении указанных лимитов рекомендуется рассмотреть переход на коммерческий тариф либо интеграцию с альтернативным геокодером.

Среди перспективных направлений развития системы можно выделить:

- реализацию фоновых уведомлений о наступлении временного окна следующей задачи;
- поддержку командной работы с общим пулом задач;
- интеграцию с внешними календарями (Google Calendar, Яндекс Календарь);
- расширение алгоритма оптимизации более мощными метаэвристиками (муравьиный алгоритм, генетический алгоритм) для повышения качества решений при большом числе задач;
- учет реального дорожного трафика при построении матрицы расстояний.

3.4 Выводы

В настоящей главе рассмотрены вопросы внедрения и опытной эксплуатации разработанного приложения. Выполнен анализ соответствия реализованной программной системы функциональным и нефункциональным требованиям, сформулированным в первой главе, и подтверждено, что все ключевые требования выполнены: реализованы регистрация пользователей и защищенная аутентификация, ведение списка задач с временными окнами, автоматическое построение оптимальной последовательности их выполнения, визуализация маршрута на карте, импорт и экспорт списка задач.

Описаны принятые архитектурные решения, обеспечивающие отказоустойчивость и надежность работы системы: использование контейнеризации, применение обратимых миграций базы данных, автоматическое восстановление сессии пользователя после истечения срока действия токена доступа, а также защищенное хранение паролей и обработка ошибок взаимодействия с внешними сервисами. По результатам нагрузочного тестирования

установлено, что система обеспечивает приемлемые показатели производительности при типовой нагрузке, а 95-й процентиль времени отклика не превышает заданного порога.

Подготовлены и описаны процедуры развертывания, резервного копирования и обновления, обеспечивающие штатное сопровождение системы в продуктивной среде. Определены текущие ограничения реализации, связанные с условиями лицензирования внешних картографических сервисов, а также сформулированы перспективные направления дальнейшего развития, среди которых интеграция с внешними календарями, поддержка совместной работы и применение более мощных метаэвристик при большом числе задач.

Полученные результаты подтверждают практическую применимость разработанного решения и его готовность к опытной эксплуатации.

ЗАКЛЮЧЕНИЕ

В ходе выполнения выпускной квалификационной работы разработано веб-приложение для автоматизированного планирования маршрутов с учетом временных окон.

В результате работы выполнены следующие задачи:

1. Проведен анализ предметной области: выявлены ограничения существующих приложений для управления задачами и навигационных сервисов, обоснована актуальность разработки.

2. Исследованы алгоритмические подходы к решению задачи маршрутизации с временными окнами; выбрана и реализована эвристика ближайшего соседа с одношаговым просмотром вперед (NNH-TW) с временной сложностью $O(n^3)$.

3. Разработана архитектура программной системы на основе принципов чистой архитектуры с разделением на слои: доменная логика, варианты использования, HTTP-доставка и репозитории баз данных.

4. Реализована серверная часть на языке Go с использованием фреймворка Gin, обеспечивающая REST API для управления задачами, маршрутами и аутентификацией пользователей. Алгоритм оптимизации поддерживает фиксацию начальной и конечной точки маршрута, а также попарные ограничения порядка посещения.

5. Разработан пользовательский интерфейс на React с отображением задач в виде карточек, интерактивной картой Яндекс JS API, выбором режима транспорта, возможностью ручного упорядочивания методом перетаскивания, импортом задач из CSV и Excel, экспортом в CSV и XLSX, а также визуальным обнаружением конфликтов временных окон.

6. Реализована интеграция с сервисом «Яндекс JavaScript API и HTTP Геокодер» версии 2.1 для геокодирования адресов с автодополнением и визу-

ализации маршрута, а также с OSRM для формирования матрицы расстояний в качестве резервного источника.

7. Проведено тестирование разработанного решения: реализовано 35 модульных тестов алгоритма оптимизации, тесты бизнес-логики и HTTP-обработчиков серверной части, тесты модулей клиентской части, обеспечивающих взаимодействие с программным интерфейсом серверной части.

8. Выполнена оценка производительности и масштабируемости системы: подтверждено соответствие нефункциональным требованиям по времени ответа, надежности и устойчивости к ограничениям внешних API.

Поставленная цель достигнута: разработанное веб-приложение устраняет выявленный в ходе анализа разрыв между приложениями для управления задачами и навигационными сервисами, предоставляя пользователю единое программное средство для автоматизированного планирования маршрутов с учетом географического положения и временных ограничений.

Перспективными направлениями развития системы являются: поддержка командной работы с общим пулом задач, интеграция с внешними календарями, реализация фоновых уведомлений о наступлении временных окон, а также внедрение более мощных метаэвристик для повышения качества маршрутов при большом числе задач.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Бондаренко Т. В., Гарибов А. И., Федотов Е. А. Решение задачи планирования маршрутов с учетом временных окон [Электронный ресурс] // Инженерный вестник Дона. — 2020. — № 5 (65). — URL: <https://cyberleninka.ru/article/n/reshenie-zadachi-planirovaniya-marshrutov-s-uchyotom-vremennyh-okon> (дата обращения: 27.02.2026).
2. The Go Programming Language [Электронный ресурс]. — Режим доступа: <https://go.dev/>, свободный (дата обращения: 15.03.2026).
3. PostgreSQL: The World's Most Advanced Open Source Relational Database [Электронный ресурс]. — Режим доступа: <https://www.postgresql.org/docs/>, свободный (дата обращения: 15.03.2026).
4. PostgreSQL Documentation. GiST Indexes [Электронный ресурс]. — Режим доступа: <https://www.postgresql.org/docs/current/gist.html>, свободный (дата обращения: 15.03.2026).
5. React — The library for web and native user interfaces [Электронный ресурс]. — Режим доступа: <https://react.dev/>, свободный (дата обращения: 15.03.2026).
6. Docker: Accelerated Container Application Development [Электронный ресурс]. — Режим доступа: <https://www.docker.com/>, свободный (дата обращения: 15.03.2026).
7. Relog — сервис оптимизации маршрутов [Электронный ресурс]. — Режим доступа: <https://relog.ru/>, свободный (дата обращения: 20.03.2026).
8. Яндекс Маршрутизация — сервис автоматического построения маршрутов [Электронный ресурс]. — Режим доступа: <https://routing.yandex.ru/>, свободный (дата обращения: 20.03.2026).
9. Spoke — Route Optimization Software [Электронный ресурс]. — Режим доступа: <https://www.spokeapp.io/>, свободный (дата обращения: 20.03.2026).

10. Гвоздев Л. Р., Медведева Т. А. Решение задачи маршрутизации транспортных средств с временными окнами с помощью алгоритма муравьиных колоний [Электронный ресурс] // Молодой исследователь Дона. — 2022. — № 3 (36). — URL: <https://cyberleninka.ru/article/n/reshenie-zadachi-marshrutizatsii-transportnyh-sredstv-s-vremennymi-oknami-s-pomoschyu-algoritma-muravinyh-koloniy> (дата обращения: 01.03.2026).

11. Hertrich C., Hungerländer P., Truden C. Sweep Algorithms for the Capacitated Vehicle Routing Problem with Structured Time Windows // Operations Research Proceedings 2018. — Cham: Springer, 2019. — P. 127–133. — DOI: 10.1007/978-3-030-18500-8_17.

12. Iklassov Z., Sobirov I., Solozabal R., Takáč M. Reinforcement Learning for Solving Stochastic Vehicle Routing Problem with Time Windows // Proceedings of the 15th Asian Conference on Machine Learning (ACML 2023). — PMLR, 2024. — Vol. 222. — P. 1551–1565.

13. Alessandretti L., Szell M. Urban Mobility // Compendium of Urban Complexity. — Cham: Springer, 2025. — P. 57–76. — DOI: 10.1007/978-3-031-82666-5_4.

14. Saioc G.-V., Shirchenko D., Chabbi M. Unveiling and Vanquishing Goroutine Leaks in Enterprise Microservices: A Dynamic Analysis Approach // Proceedings of the IEEE/ACM 46th International Conference on Software Engineering: Companion (ICSE-Companion 2024). — New York: ACM, 2024. — P. 314–325. — DOI: 10.1145/3639478.3640033.

15. Jones M., Bradley J., Sakimura N. JSON Web Token (JWT): RFC 7519 [Электронный ресурс]. — Internet Engineering Task Force, 2015. — Режим доступа: <https://www.rfc-editor.org/rfc/rfc7519>, свободный (дата обращения: 09.04.2026).

16. Provos N., Mazieres D. A Future-Adaptable Password Scheme // Proceedings of the FREENIX Track: 1999 USENIX Annual Technical Conference. — Monterey: USENIX Association, 1999. — P. 81–91.

17. Мартин Р. К. Чистая архитектура: искусство разработки программного обеспечения. — Санкт-Петербург: Питер, 2018. — 352 с. — (Серия «Библиотека программиста»). — ISBN 978-5-4461-0772-8.
18. Held M., Karp R. M. A Dynamic Programming Approach to Sequencing Problems // Journal of the Society for Industrial and Applied Mathematics. — 1962. — Vol. 10, No. 1. — P. 196–210. — DOI: 10.1137/0110015.
19. Гладков Л. А., Курейчик В. В., Курейчик В. М. Генетические алгоритмы: учебник / под ред. В. М. Курейчика. — 2-е изд., испр. и доп. — Москва: ФИЗМАТЛИТ, 2010. — 368 с. — ISBN 978-5-9221-0510-1.
20. Project OSRM — Open Source Routing Machine [Электронный ресурс]. — Режим доступа: <https://project-osrm.org/>, свободный (дата обращения: 11.04.2026).
21. Luxen D., Vetter C. Real-time Routing with OpenStreetMap Data // Proceedings of the 19th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems (GIS '11). — New York: ACM, 2011. — P. 513–516. — DOI: 10.1145/2093973.2094062.
22. Яндекс JavaScript API и HTTP Геокодер [Электронный ресурс]. — Режим доступа: <https://yandex.ru/dev/jsapi-v2-1/doc/ru/>, свободный (дата обращения: 12.04.2026).
23. PostGIS: Spatial and Geographic Objects for PostgreSQL [Электронный ресурс]. — Режим доступа: <https://postgis.net/>, свободный (дата обращения: 11.04.2026).
24. Gin Web Framework [Электронный ресурс]. — Режим доступа: <https://gin-gonic.com/>, свободный (дата обращения: 11.04.2026).
25. TypeScript: JavaScript With Syntax For Types [Электронный ресурс]. — Режим доступа: <https://www.typescriptlang.org/>, свободный (дата обращения: 11.04.2026).
26. Vite: Next Generation Frontend Tooling [Электронный ресурс]. — Режим доступа: <https://vite.dev/>, свободный (дата обращения: 11.04.2026).

27. Bogner J., Merkel M. To Type or Not to Type? A Systematic Comparison of the Software Quality of JavaScript and TypeScript Applications on GitHub // Proceedings of the 19th International Conference on Mining Software Repositories (MSR 2022). — New York: ACM, 2022. — P. 658–669. — DOI: 10.1145/3524842.3528454.
28. Liu F., Lu C., Gui L., Zhang Q., Tong X., Yuan M. Heuristics for Vehicle Routing Problem: A Survey and Recent Advances [Электронный ресурс]. — 2023. — Режим доступа: <https://arxiv.org/abs/2303.04147>, свободный (дата обращения: 12.04.2026).
29. Solomon M. M. Algorithms for the Vehicle Routing and Scheduling Problems with Time Window Constraints // Operations Research. — 1987. — Vol. 35, No. 2. — P. 254–265. — DOI: 10.1287/opre.35.2.254.
30. Кормен Т., Лейзерсон Ч., Ривест Р., Штайн К. Алгоритмы: построение и анализ. — 3-е изд. — Москва: Вильямс, 2013. — 1328 с. — ISBN 978-5-8459-1794-2.
31. ГОСТ Р ИСО/МЭК 25010-2015. Информационные технологии. Системная и программная инженерия. Требования и оценка качества систем и программного обеспечения (SQuaRE). Модели качества систем и программных продуктов. — Москва: Стандартинформ, 2015. — 36 с.
32. ГОСТ 19.701-90 (ИСО 5807-85). Единая система программной документации. Схемы алгоритмов, программ, данных и систем. Обозначения условные и правила выполнения. — Москва: Стандартинформ, 1992. — 26 с.
33. Куликов С. С. Тестирование программного обеспечения. Базовый курс. — 2-е изд. — Минск: Четыре четверти, 2017. — 312 с. — ISBN 978-985-7081-77-0.
34. Hardt D. The OAuth 2.0 Authorization Framework: RFC 6749 [Электронный ресурс]. — Internet Engineering Task Force, 2012. — Режим доступа: <https://www.rfc-editor.org/rfc/rfc6749>, свободный (дата обращения: 26.04.2026).

35. Девицына С. Н., Пилькевич П. В., Удод Е. В. Способы улучшения защищенности сервисов, использующих JWT-токены // Экономика. Информатика. — 2023. — Т. 50, № 1. — С. 144–151. — DOI: 10.52575/2687-0932-2023-50-1-144-151.

36. Кудухов А. Н. К вопросу о применении алгоритмов замещения LRU в подсистемах кэширования информационных систем промышленных предприятий [Электронный ресурс] // Перспективы развития информационных технологий. — 2014. — URL: <https://cyberleninka.ru/article/n/k-voprosu-o-primenenii-algoritmov-zamescheniya-lru-v-podsistemah-keshirovaniya-informatsionnyh-sistem-promyshlennyh-predpriyatiy> (дата обращения: 26.04.2026).

37. Коптева А. В., Князев И. В. Анализ чистой архитектуры Golang REST API с внедрением зависимостей, следуя принципам SOLID [Электронный ресурс] // Проблемы современной науки и образования. — 2021. — URL: <https://cyberleninka.ru/article/n/analiz-chistoy-arhitektury-golang-rest-api-s-vnedreniem-zavisimostey-sleduya-printsipam-solid> (дата обращения: 26.04.2026).

38. Бронникова Е. М. Использование информационных технологий в тайм-менеджменте [Электронный ресурс] // Научные труды Вольного экономического общества России. — 2010. — URL: <https://cyberleninka.ru/article/n/ispolzovanie-informatsionnyh-tehnologiy-v-taymmenedzhmente> (дата обращения: 26.04.2026).

39. Шкодинский С. В., Козлова А. Р. Ключевые аспекты логистики последней мили в онлайн-торговле // Прикладные экономические исследования. — 2025. — № 5. — С. 223–228. — DOI: 10.47576/2949-1908.2025.5.5.026.

40. Necula R., Breaban M., Raschip M. Tackling Dynamic Vehicle Routing Problem with Time Windows by means of Ant Colony System // Proceedings of the 2017 IEEE Congress on Evolutionary Computation (CEC 2017). — Donostia: IEEE, 2017. — P. 2480–2487. — DOI: 10.1109/CEC.2017.7969606.

41. Grafana Labs. k6: Modern load testing for developers [Электронный ресурс]. — Режим доступа: <https://k6.io/>, свободный (дата обращения:

12.04.2026).

42. Todoist: Organize your work and life, finally [Электронный ресурс]. — Режим доступа: <https://todoist.com/pricing>, свободный (дата обращения: 12.04.2026).

43. Microsoft To Do: Tasks, To-Do lists & Reminders [Электронный ресурс]. — Режим доступа: <https://to-do.microsoft.com/>, свободный (дата обращения: 12.04.2026).

44. Google Maps [Электронный ресурс]. — Режим доступа: <https://maps.google.com/>, свободный (дата обращения: 12.04.2026).

45. Google Maps Help. Getting directions and showing routes [Электронный ресурс]. — Режим доступа: <https://support.google.com/maps/answer/144339>, свободный (дата обращения: 12.04.2026).

46. Routific: Intelligent Route Optimization Software [Электронный ресурс]. — Режим доступа: <https://routific.com/pricing>, свободный (дата обращения: 12.04.2026).

47. HashiCorp. golang-lru: Golang LRU cache [Электронный ресурс]. — Режим доступа: <https://github.com/hashicorp/golang-lru>, свободный (дата обращения: 12.04.2026).

48. Авдошин С. М., Береснева Е. Н. Эвристические методы конструирования маршрута для решения задачи маршрутизации с ограничением по грузоподъемности [Электронный ресурс] // Труды Института системного программирования РАН. — 2018. — URL: <https://ispranproceedings.elpub.ru/jour/article/view/1184> (дата обращения: 26.04.2026).

49. Гончаров Е. Н., Захарова А. А. Обзор динамических задач маршрутизации транспорта [Электронный ресурс] // Управление большими системами. — 2021. — URL: <https://cyberleninka.ru/article/n/obzor-dinamicheskikh-zadach-marshrutizatsii-transporta> (дата обращения: 26.04.2026).

50. Левкин Г. Г., Куликова О. М. Аспекты управления логистикой на этапе последней мили [Электронный ресурс] // Вестник евразийской науки.

— 2021. — URL: <https://cyberleninka.ru/article/n/aspekty-upravleniya-logistik-ou-na-etape-posledney-mili> (дата обращения: 26.04.2026).

51. Кнут Д. Э. Искусство программирования. Том 1. Основные алгоритмы. — 3-е изд. — Москва: Вильямс, 2018. — 720 с. — ISBN 978-5-907144-12-1.

52. Дейт К. Дж. Введение в системы баз данных. — 8-е изд. — Москва: Вильямс, 2019. — 1328 с. — ISBN 978-5-8459-0788-2.

53. Гамма Э., Хелм Р., Джонсон Р., Влиссидес Дж. Приемы объектно-ориентированного проектирования. Паттерны проектирования. — Санкт-Петербург: Питер, 2020. — 368 с. — (Серия «Библиотека программиста»). — ISBN 978-5-496-00389-6.

54. Фаулер М. Рефакторинг: улучшение существующего кода. — 2-е изд. — Москва: Диалектика, 2020. — 448 с. — ISBN 978-5-907144-83-1.

55. Эванс Э. Предметно-ориентированное проектирование: структуризация сложных программных систем. — Москва: Вильямс, 2019. — 448 с. — ISBN 978-5-907114-52-0.

56. Хант Э., Томас Д. Программист-прагматик. Путь от подмастерья к мастеру. — 2-е изд. — Москва: Бомбора, 2020. — 320 с. — ISBN 978-5-04-110494-0.

57. Бек К. Экстремальное программирование: разработка через тестирование. — Санкт-Петербург: Питер, 2017. — 224 с. — ISBN 978-5-496-02570-6.

58. Майерс Г., Сандлер К., Баджетт Т. Искусство тестирования программ. — 3-е изд. — Москва: Диалектика, 2018. — 272 с. — ISBN 978-5-8459-1796-6.

59. ГОСТ 7.32-2017. Система стандартов по информации, библиотечному и издательскому делу. Отчет о научно-исследовательской работе. Структура и правила оформления. — Москва: Стандартинформ, 2017. — 27 с.

60. ГОСТ Р 7.0.100-2018. Система стандартов по информации, библиотечному и издательскому делу. Библиографическая запись. Библиографическое описание. Общие требования и правила составления. — Москва: Стандартинформ, 2018. — 65 с.

61. ГОСТ Р ИСО/МЭК 12207-2010. Информационная технология. Системная и программная инженерия. Процессы жизненного цикла программных средств. — Москва: Стандартинформ, 2011. — 95 с.

62. Мартин Р. С. Чистый код: создание, анализ и рефакторинг. — Санкт-Петербург: Питер, 2019. — 464 с. — (Серия «Библиотека программиста»). — ISBN 978-5-4461-0960-8.

63. Орлов С. А. Технологии разработки программного обеспечения: учебник для вузов. — 4-е изд. — Санкт-Петербург: Питер, 2018. — 608 с. — ISBN 978-5-4461-1213-4.

64. Ехлаков Ю. П. Управление программными проектами: учебник. — Томск: ТУСУР, 2020. — 244 с. — ISBN 978-5-4332-0152-8.

65. Долгова О. Э., Пересветов В. В. Муравьиный алгоритм с ослаблением ограничений по временным окнам в решении задачи маршрутизации транспорта // Вычислительные технологии. — 2018. — Т. 23, № 5. — С. 30–43. — DOI: 10.25743/ICT.2018.23.16886.

66. Хачай М. Ю., Огородников Ю. Ю. Полиномиальная приближенная схема для задачи маршрутизации транспортных средств с ограничениями на грузоподъемность и временные промежутки обслуживания // Труды Института математики и механики УрО РАН. — 2018. — Т. 24, № 3. — С. 233–246. — DOI: 10.21538/0134-4889-2018-24-3-233-246.

ПРИЛОЖЕНИЕ А

(справочное)

Схема базы данных и конфигурация контейнеризации

Схема базы данных (SQL-миграции)

Листинг А.1 – Миграция 0001_init.up.sql — инициализация схемы базы данных

```
1 -- Включаем расширения (UUID и PostGIS)
2 CREATE EXTENSION IF NOT EXISTS "uuid-osspl";
3 CREATE EXTENSION IF NOT EXISTS "postgis";

4 -- users
5 CREATE TABLE IF NOT EXISTS users (
6     id                UUID                PRIMARY KEY DEFAULT
    ↪  uuid_generate_v4(),
7     email             VARCHAR(255) NOT NULL UNIQUE,
8     password_hash     VARCHAR(255) NOT NULL,
9     created_at        TIMESTAMPTZ NOT NULL DEFAULT now(),
10    updated_at         TIMESTAMPTZ NOT NULL DEFAULT now()
11 );

12 -- refresh_tokens
13 CREATE TABLE IF NOT EXISTS refresh_tokens (
14     id                UUID                PRIMARY KEY DEFAULT
    ↪  uuid_generate_v4(),
15     user_id           UUID                NOT NULL REFERENCES users(id) ON
    ↪  DELETE CASCADE,
16     token_hash        VARCHAR(255) NOT NULL UNIQUE,
17     expires_at        TIMESTAMPTZ NOT NULL,
18     revoked_at        TIMESTAMPTZ NULL,
19     created_at        TIMESTAMPTZ NOT NULL DEFAULT now()
```



```

20 );
21 CREATE INDEX IF NOT EXISTS refresh_tokens_user_id_idx
22     ON refresh_tokens(user_id);

23 -- tasks
24 CREATE TABLE IF NOT EXISTS tasks (
25     id            UUID            PRIMARY KEY DEFAULT
    ↪ uuid_generate_v4(),
26     user_id       UUID            NOT NULL REFERENCES users(id)
    ↪ ON DELETE CASCADE,
27     title         VARCHAR(200) NOT NULL,
28     address_text  VARCHAR(400) NOT NULL,
29     latitude      NUMERIC(9,6) NULL,
30     longitude     NUMERIC(9,6) NULL,
31     duration_min  INT            NOT NULL,
32     window_start  TIME          NULL,
33     window_end    TIME          NULL,
34     sort_index    INT            NOT NULL DEFAULT 0,
35     created_at    TIMESTAMPTZ    NOT NULL DEFAULT now(),
36     updated_at    TIMESTAMPTZ    NOT NULL DEFAULT now(),
37     is_deleted    BOOLEAN        NOT NULL DEFAULT false
38 );
39 CREATE INDEX IF NOT EXISTS tasks_user_id_idx
40     ON tasks(user_id);
41 CREATE INDEX IF NOT EXISTS tasks_user_sort_idx
42     ON tasks(user_id, sort_index);

43 -- routes
44 CREATE TABLE IF NOT EXISTS routes (
45     id            UUID            PRIMARY KEY DEFAULT
    ↪ uuid_generate_v4(),
46     user_id       UUID            NOT NULL REFERENCES users(id) ON
    ↪ DELETE CASCADE,

```

```

47     status          VARCHAR(30) NOT NULL,      --
    ↪     draft|optimized|failed
48     source          VARCHAR(30) NOT NULL,      -- manual|optimized
49     algorithm       VARCHAR(50) NULL,
50     started_at      TIMESTAMPTZ NULL,
51     finished_at     TIMESTAMPTZ NULL,
52     created_at      TIMESTAMPTZ NOT NULL DEFAULT now()
53 );
54 CREATE INDEX IF NOT EXISTS routes_user_id_idx ON
    ↪     routes(user_id);

55 -- route_stats (1:1 κ routes)
56 CREATE TABLE IF NOT EXISTS route_stats (
57     route_id         UUID PRIMARY KEY REFERENCES routes(id)
    ↪     ON DELETE CASCADE,
58     total_distance_m INT NULL,
59     total_travel_sec INT NULL,
60     total_service_sec INT NULL,
61     total_wait_sec   INT NULL,
62     computed_at      TIMESTAMPTZ NOT NULL DEFAULT now()
63 );

64 -- route_stops
65 CREATE TABLE IF NOT EXISTS route_stops (
66     id                UUID PRIMARY KEY DEFAULT
    ↪     uuid_generate_v4(),
67     route_id          UUID NOT NULL REFERENCES routes(id)
    ↪     ON DELETE CASCADE,
68     task_id           UUID NOT NULL REFERENCES tasks(id)
    ↪     ON DELETE RESTRICT,
69     position          INT NOT NULL,
70     travel_from_prev_sec INT NULL,
71     arrive_time       TIMESTAMPTZ NULL,
72     service_start_time TIMESTAMPTZ NULL,

```

```

73     service_end_time    TIMESTAMPTZ NULL,
74     wait_sec            INT NULL
75 );
76 CREATE INDEX IF NOT EXISTS route_stops_route_id_idx
77     ON route_stops(route_id);
78 CREATE UNIQUE INDEX IF NOT EXISTS
    ↪ route_stops_route_position_uq
79     ON route_stops(route_id, position);

80 -- route_geometry (1:1 к routes)
81 CREATE TABLE IF NOT EXISTS route_geometry (
82     route_id    UUID PRIMARY KEY REFERENCES routes(id) ON
    ↪ DELETE CASCADE,
83     polyline    TEXT NOT NULL,
84     bbox_json   JSONB NULL,
85     geojson_json JSONB NULL,
86     updated_at  TIMESTAMPTZ NOT NULL DEFAULT now()
87 );

```

Листинг A.2 – Миграция 0002_routes_name.up.sql — добавление поля name

```

1 ALTER TABLE routes ADD COLUMN IF NOT EXISTS name
    ↪ VARCHAR(200) NULL;

```

Листинг A.3 – Миграция 0003_task_completed.up.sql — добавление признака выполнения

```

1 ALTER TABLE tasks ADD COLUMN IF NOT EXISTS is_completed
    ↪ BOOLEAN NOT NULL DEFAULT false;

```

Листинг A.4 – Миграция 0004_task_duration_optional.up.sql — опциональная длительность

```
1 ALTER TABLE tasks ALTER COLUMN duration_min DROP NOT NULL;
```

Листинг А.5 – Миграция 0005_task_datetime_windows.up.sql — отдельные дата и время окна

```
1 -- Разделяем window_start/window_end (TIME) на отдельные
   ↳ колонки дата + время
2 ALTER TABLE tasks
3     DROP COLUMN window_start,
4     DROP COLUMN window_end;
5 ALTER TABLE tasks
6     ADD COLUMN window_start_date DATE NULL,
7     ADD COLUMN window_start_time TIME NULL,
8     ADD COLUMN window_end_date DATE NULL,
9     ADD COLUMN window_end_time TIME NULL;
```

Конфигурация контейнеризации

Листинг А.6 – Конфигурация docker-compose.yml серверной части проекта

```
1 services:
2   db:
3     image: postgis/postgis:16-3.4
4     environment:
5       POSTGRES_DB: planner
6       POSTGRES_USER: planner
7       POSTGRES_PASSWORD: planner
8     ports:
9       - "5432:5432"
10    volumes:
```

```

11     - db_data:/var/lib/postgresql/data
12     healthcheck:
13         test: ["CMD-SHELL", "pg_isready -U planner -d
↪ planner"]
14         interval: 5s
15         timeout: 3s
16         retries: 20
17     migrate:
18         image: migrate/migrate:v4.17.1
19         depends_on:
20             db:
21                 condition: service_healthy
22         volumes:
23             - ./migrations:/migrations
24         entrypoint:
25             - "migrate"
26             - "-path"
27             - "/migrations"
28             - "--database"
29             - "postgres://planner:planner@db:5432/planner?sslmode=
↪ disable"
30             - "up"
31     api:
32         build: .
33         depends_on:
34             db:
35                 condition: service_healthy
36             migrate:
37                 condition: service_completed_successfully
38         environment:
39             APP_HTTP_ADDR: ":8080"

```

```
40     APP_DB_DSN: "postgres://planner:planner@db:5432/planner?sslmode=disable"
↩
41     APP_JWT_SECRET: "change-me-super-secret"
42     APP_ACCESS_TTL_MIN: "15"
43     APP_REFRESH_TTL_DAYS: "30"
44     APP_CORS_ORIGINS: "http://localhost:5173"
45     ports:
46     - "8080:8080"

47 volumes:
48     db_data:
```

ПРИЛОЖЕНИЕ Б

(справочное)

Листинги ключевых фрагментов кода

Листинг Б.1 – Алгоритм ближайшего соседа с временными окнами и просмотром вперед (nearest_neighbor.go)

```
1 package optimizer

2 import (
3     "context"
4     "math"
5 )

6 // NearestNeighborTW – эвристика ближайшего соседа с учетом
   ↪ временных окон
7 // (NNH-TW). Все временные величины хранятся в секундах
   ↪ Unix. Сложность –  $O(n^3)$ .
8 type NearestNeighborTW struct{}

9 func NewNearestNeighborTW() *NearestNeighborTW { return
   ↪ &NearestNeighborTW{} }
10 func (a *NearestNeighborTW) Name() string { return
   ↪ "nearest-neighbor-tw" }

11 func (a *NearestNeighborTW) Optimize(
12     _ context.Context, g *Graph, startTimeUnix int64, c
   ↪ Constraints,
13 ) (Result, error) {
14     n := len(g.Nodes)
15     if n == 0 { return Result{}, nil }

16     prereqs := buildPrereqs(n, c.PrecedencePairs)
17     endIdx := -1
```

```

18     if c.EndNodeIdx != nil { endIdx = *c.EndNodeIdx }

19     visited := make([]bool, n)
20     order := make([]int, 0, n)

21     var cur int
22     if c.StartNodeIdx != nil {
23         cur = *c.StartNodeIdx
24     } else {
25         cur = startNode(g, prereqs, endIdx)
26     }
27     visited[cur] = true
28     order = append(order, cur)
29     currentTimeSec := startTimeUnix +
↪ int64(g.Nodes[cur].DurationMin)*60

30     var totalDistM, totalTravelSec, totalServiceSec,
↪ totalWaitSec int
31     totalServiceSec = g.Nodes[cur].DurationMin * 60

32     for len(order) < n {
33         // Закрепленный конечный узел — ставим последним.
34         if endIdx >= 0 && !visited[endIdx] {
35             allOther := true
36             for i := 0; i < n; i++ {
37                 if !visited[i] && i != endIdx { allOther =
↪ false; break }
38             }
39             if allOther {
40                 currentTimeSec, totalDistM, totalTravelSec,
↪ totalServiceSec,
41                 totalWaitSec, _ = appendNode(endIdx,
↪ cur, currentTimeSec, g,

```



```

42         totalDistM, totalTravelSec,
↪     totalServiceSec, totalWaitSec)
43         visited[endIdx] = true; order =
↪     append(order, endIdx); break
44     }
45 }

46     next := -1
47     var bestComp int64 = math.MaxInt64
48     var bestDeadline int64 = math.MaxInt64
49     bestSafe := false

50     // Проход 1: допустимый узел + look-ahead.
51     for i := 0; i < n; i++ {
52         if visited[i] || i == endIdx { continue }
53         if !prereqsMet(i, visited, prereqs) { continue }
54         arrival := currentTimeSec +
↪     int64(g.Edges[cur][i].DurationSec)
55         if !feasible(g.Nodes[i], arrival) { continue }
56         comp := nodeCompletionTime(g.Nodes[i], arrival)
57         deadline := g.Nodes[i].WindowEnd
58         if deadline < 0 { deadline = math.MaxInt64 }
59         safe := !causesWindowMiss(i, comp, g, visited,
↪     endIdx)
60         better := false
61         switch {
62             case safe && !bestSafe: better = true
63             case safe == bestSafe:
64                 better = comp < bestComp ||
65                 (comp == bestComp && deadline <
↪     bestDeadline)
66         }
67         if better {
68             bestComp = comp; bestDeadline = deadline

```

```

69         bestSafe = safe; next = i
70     }
71 }

72     // Проход 2: fallback – без учета окна.
73     if next == -1 {
74         bestComp = math.MaxInt64
75         for i := 0; i < n; i++ {
76             if visited[i] || i == endIdx { continue }
77             if !prereqsMet(i, visited, prereqs) {
↪ continue }
78             arrival := currentTimeSec +
↪ int64(g.Edges[cur][i].DurationSec)
79             comp := nodeCompletionTime(g.Nodes[i],
↪ arrival)
80             if comp < bestComp { bestComp = comp; next =
↪ i }
81         }
82     }
83     if next == -1 { break }

84     currentTimeSec, totalDistM, totalTravelSec,
↪ totalServiceSec,
85     totalWaitSec, _ = appendNode(next, cur,
↪ currentTimeSec, g,
86     totalDistM, totalTravelSec, totalServiceSec,
↪ totalWaitSec)
87     visited[next] = true; order = append(order, next);
↪ cur = next
88 }
89 return Result{Order: order, TotalDistanceM: totalDistM,
90     TotalTravelSec: totalTravelSec, TotalServiceSec:
↪ totalServiceSec,
91     TotalWaitSec: totalWaitSec}, nil

```

```

92 }

93 func feasible(node Node, arrivalSec int64) bool {
94     if node.WindowEnd < 0 { return true }
95     return arrivalSec <=
        ↪ node.WindowEnd-int64(node.DurationMin)*60
96 }

97 // causesWindowMiss – look-ahead: true, если после cand
98 // хотя бы один узел с окном становится недостижимым.
99 func causesWindowMiss(cand int, afterSec int64,
100     g *Graph, visited []bool, endIdx int) bool {
101     for j := 0; j < len(g.Nodes); j++ {
102         if visited[j] || j == cand || j == endIdx { continue
        ↪ }
103         if g.Nodes[j].WindowEnd < 0 { continue }
104         arr := afterSec +
        ↪ int64(g.Edges[cand][j].DurationSec)
105         if !feasible(g.Nodes[j], arr) { return true }
106     }
107     return false
108 }

```

Листинг Б.2 – HTTP-обработчик оптимизации маршрута
(handlers_routes.go, метод Optimize)

```

1 // Optimize – POST /api/routes/optimize.
2 func (h *RouteHandlers) Optimize(c *gin.Context) {
3     userID, ok := userIDFromCtx(c)
4     if !ok { return }

5     var req OptimizeRouteReq
6     if err := c.ShouldBindJSON(&req); err != nil {

```

```

7         c.JSON(http.StatusBadRequest, gin.H{"error": "bad
↪ request"})
8         return
9     }

10     startTime := req.StartTimeUnix
11     if startTime <= 0 { startTime = time.Now().Unix() }

12     var matrix [][]distance.Edge
13     if len(req.DistanceMatrix) > 0 {
14         matrix = make([][]distance.Edge,
↪ len(req.DistanceMatrix))
15         for i, row := range req.DistanceMatrix {
16             matrix[i] = make([]distance.Edge, len(row))
17             for j, cell := range row {
18                 matrix[i][j] = distance.Edge{
19                     DistanceM: cell.DistanceM, DurationSec:
↪ cell.DurationSec,
20                     }
21             }
22         }
23     }

24     precedences := make([]routestory.PrecedenceConstraint,
25         len(req.PrecedenceConstraints))
26     for i, p := range req.PrecedenceConstraints {
27         precedences[i] = routestory.PrecedenceConstraint{
28             BeforeTaskID: p.BeforeTaskID, AfterTaskID:
↪ p.AfterTaskID,
29         }
30     }

31     out, err := h.story.Optimize(c.Request.Context(),
↪ userID,

```

```

32         req.TaskIDs, startTime, matrix,
33         req.StartTaskID, req.EndTaskID, precedences)
34     if err != nil {
35         c.JSON(http.StatusBadRequest, gin.H{"error":
↵ err.Error()})
36         return
37     }
38     c.JSON(http.StatusOK, out)
39 }

40 // OptimizeRouteReq – тело POST /api/routes/optimize.
41 type OptimizeRouteReq struct {
42     TaskIDs                []string
↵     `json:"taskIds"`
43     StartTimeUnix          int64
↵     `json:"startTimeUnix"`
44     DistanceMatrix          [][]DistanceCellDTO
↵     `json:"distanceMatrix,omitempty"`
45     StartTaskID             *string
↵     `json:"startTaskId,omitempty"`
46     EndTaskID               *string
↵     `json:"endTaskId,omitempty"`
47     PrecedenceConstraints []PrecedencePairDTO
↵     `json:"precedenceConstraints,omitempty"`
48 }

```

Листинг Б.3 – Ключевые функции API-клиента: оптимизация маршрута и обновление токена (client.ts)

```

1 let refreshSessionPromise: Promise<AuthSession> | null =
↵     null;

2 export interface PrecedenceConstraint {

```

```

3   beforeTaskId: string;
4   afterTaskId: string;
5 }

6 // optimizeRoute передает идентификаторы задач, матрицу
   ↳ расстояний
7 // и дополнительные ограничения на бэкенд.
8 export async function optimizeRoute(
9   taskIds: string[],
10  options: AuthorizedRequestOptions,
11  startTimeUnix = 0,
12  distanceMatrix?: DistanceCell[][[]],
13  startTaskId?: string,
14  endTaskId?: string,
15  precedenceConstraints?: PrecedenceConstraint[],
16 ): Promise<SavedRouteDetails> {
17   const route = await requestWithAuth<ApiRouteFull>(
18     '/routes/optimize',
19     {
20       method: 'POST',
21       body: JSON.stringify({
22         taskIds, startTimeUnix, distanceMatrix,
23         startTaskId: startTaskId ?? undefined,
24         endTaskId: endTaskId ?? undefined,
25         precedenceConstraints: precedenceConstraints?.length
26           ? precedenceConstraints : undefined,
27       }),
28     },
29     options,
30   );
31   return mapRouteDetails(route);
32 }

33 // requestWithAuth выполняет HTTP-запрос с токеном доступа.

```

```

34 // При получении 401 однократно обновляет токен и повторяет
    ↪ запрос.
35 async function requestWithAuth<T>(
36   path: string, init: RequestInit,
37   options: AuthorizedRequestOptions,
38 ): Promise<T> {
39   try {
40     return await request<T>(path, init,
    ↪ options.session.accessToken);
41   } catch (error) {
42     if (!(error instanceof ApiError) || error.status !==
    ↪ 401) throw error;
43   }
44   try {
45     const nextSession = await refreshSession(options);
46     return request<T>(path, init, nextSession.accessToken);
47   } catch (error) {
48     await options.onUnauthorized();
49     throw error;
50   }
51 }

52 // refreshSession — дедупликация через единственный промис.
53 async function refreshSession(options:
    ↪ AuthorizedRequestOptions) {
54   if (!refreshSessionPromise) {
55     refreshSessionPromise =
    ↪ request<TokenPairResponse>('/auth/refresh', {
56     method: 'POST',
57     body: JSON.stringify({
58       refreshToken: options.session.refreshToken,
59     }),
60   })
61     .then((pair) => {

```

```

62         const nextSession = buildSession(pair,
63             getEmailFromToken(pair.accessToken) ??
        ↪ options.session.email);
64         options.onSessionChange(nextSession);
65         return nextSession;
66     })
67     .finally(() => { refreshSessionPromise = null; });
68 }
69 return refreshSessionPromise;
70 }

```

Листинг Б.4 – Построение матрицы расстояний через Яндекс Маршруты JS API (routeOptimizer.ts)

```

1 import { Task } from '../types/task';
2 import { loadYandexMaps } from './yandexMaps';

3 export interface DistanceCell {
4     distanceM: number;
5     durationSec: number;
6 }

7 /**
8  * Строит матрицу расстояний n×n для списка задач через
        ↪ Яндекс Маршруты.
9  * Используется тот же движок, что и для визуализации
        ↪ маршрута на карте,
10 * поэтому данные оптимизации согласованы с отображением.
11 *
12 * matrix[i][j] = стоимость проезда от tasks[i] до tasks[j].
13 * Диагональные элементы (i === j) равны нулю.
14 */
15 export async function buildYandexDistanceMatrix(

```



```

16   tasks: Task[],
17   routingMode: 'auto' | 'pedestrian' | 'masstransit' =
    ↪ 'auto',
18 ): Promise<DistanceCell[][]> {
19   const n = tasks.length;
20   if (n === 0) return [];

21   await loadYandexMaps();

22   // masstransit через multiRouter не возвращает надежные
    ↪ данные по времени —
23   // используем 'auto' как приближение.
24   const mode = routingMode === 'masstransit' ? 'auto' :
    ↪ routingMode;

25   const fallbackSec = 99_999;
26   const matrix: DistanceCell[][] = Array.from({ length: n
    ↪ }, (_, i) =>
27     Array.from({ length: n }, (_, j) =>
28       i === j
29         ? { distanceM: 0, durationSec: 0 }
30         : { distanceM: 0, durationSec: fallbackSec },
31     ),
32   );

33   // Запрашиваем все пары параллельно: n*(n-1) запросов.
34   const pairs: [number, number][] = [];
35   for (let i = 0; i < n; i++)
36     for (let j = 0; j < n; j++)
37       if (i !== j) pairs.push([i, j]);

38   await Promise.all(
39     pairs.map(async ([i, j]) => {
40       const from = tasks[i];

```

```

41     const to = tasks[j];
42     if (from.latitude == null || to.latitude == null)
↳ return;

43     try {
44         const { distanceM, durationSec } = await new
↳ Promise<{
45             distanceM: number;
46             durationSec: number;
47         }>((resolve, reject) => {
48             const mr = new (window.ymaps as
↳ any).multiRouter.MultiRoute(
49                 {
50                     referencePoints: [
51                         [from.latitude!, from.longitude!],
52                         [to.latitude!, to.longitude!],
53                     ],
54                     params: { routingMode: mode },
55                 },
56                 {},
57             );

58             mr.model.events.once('requestsuccess', () => {
59                 try {
60                     const activeRoute: any = mr.getActiveRoute();
61                     if (activeRoute) {
62                         const distObj: any =
↳ activeRoute.properties.get('distance');
63                         const durObj: any =
↳ activeRoute.properties.get('duration');
64                         resolve({
65                             distanceM: Math.round(
66                                 typeof distObj?.value === 'number' ?
↳ distObj.value : 0,

```

```

67         ),
68         durationSec: Math.round(
69             typeof durObj?.value === 'number'
70             ? durObj.value
71             : fallbackSec,
72         ),
73     });
74     return;
75 }

76     // Запасной путь: данные из модельных
↪ маршрутов напрямую.
77     const modelRoutes: any[] =
↪ mr.model.getRoutes();
78     if (modelRoutes.length > 0) {
79         const legs: any[] =
↪ modelRoutes[0].getLegs();
80         let totalDistM = 0;
81         let totalDurSec = 0;
82         legs.forEach((leg: any) => {
83             const d: any =
↪ leg.properties?.get?.('distance');
84             const dur: any =
↪ leg.properties?.get?.('duration');
85             totalDistM += typeof d?.value === 'number'
↪ ? d.value : 0;
86             totalDurSec +=
87                 typeof dur?.value === 'number' ?
↪ dur.value : 0;
88         });
89         resolve({
90             distanceM: Math.round(totalDistM),
91             durationSec:
92                 totalDurSec > 0

```

```

93         ? Math.round(totalDurSec)
94         : fallbackSec,
95     });
96     return;
97 }
98 reject(new Error('no route data'));
99 } catch (e) {
100     reject(e);
101 }
102 });

103     mr.model.events.once('requestfail', () => {
104         reject(new Error('routing failed'));
105     });
106 });

107     matrix[i][j] = { distanceM, durationSec };
108 } catch {
109     // Оставляем fallback: алгоритм избегает
    ↪ недостижимых пар.
110 }
111 },
112 );

113 return matrix;
114 }

```